

ECE/CS 757 Advanced Computer Architecture II

Spring 2025

Consistency

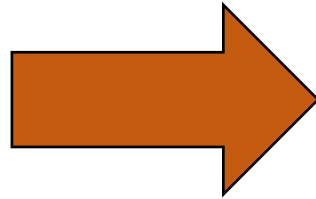
Instructor: Prof. Joshua San Miguel

Some slides adopted from Greg Steffan, Natalie Enright Jerger and Mikko Lipasti

Background – Common Subexpression Elimination

Compiler optimization to avoid redundant computations

```
a = b * c + g;  
d = b * c * e;
```

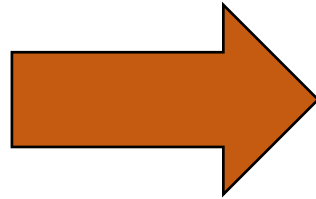


```
tmp = b * c;  
a = tmp + g;  
d = tmp * e;
```

Background – Code Hoisting

Compiler optimization to move repetitive computations out of basic blocks

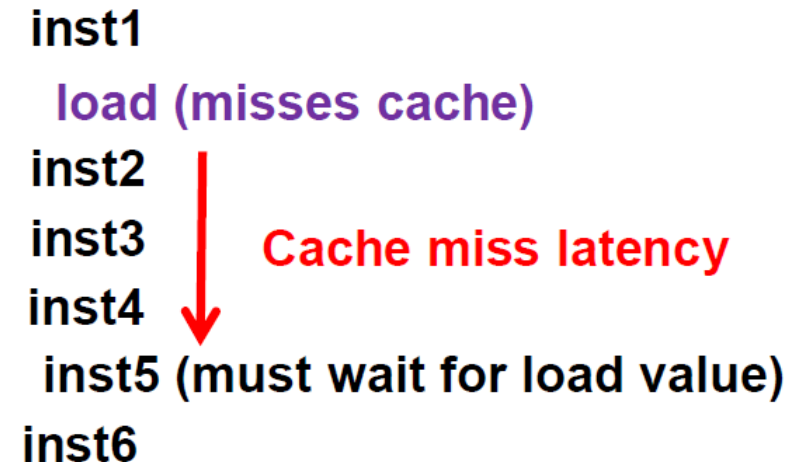
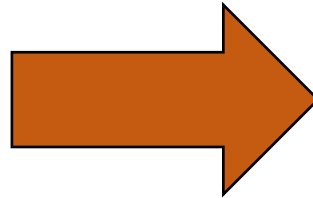
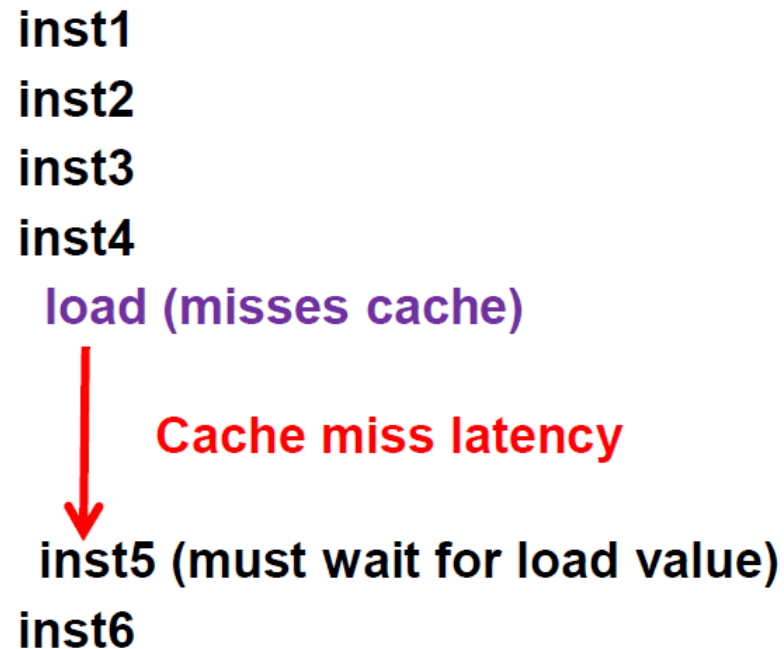
```
for(i = 0; i < N; i++)  
    a[i] = b[X];
```



```
tmp = b[X];  
for (i = 0; i < N; i++)  
    a[i] = tmp;
```

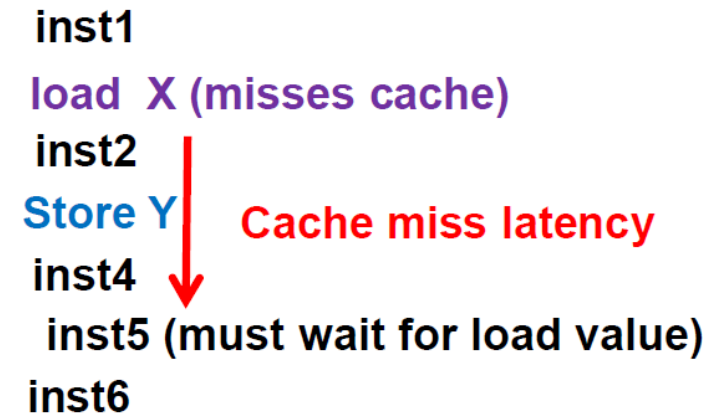
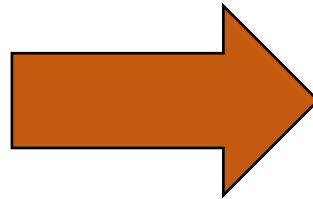
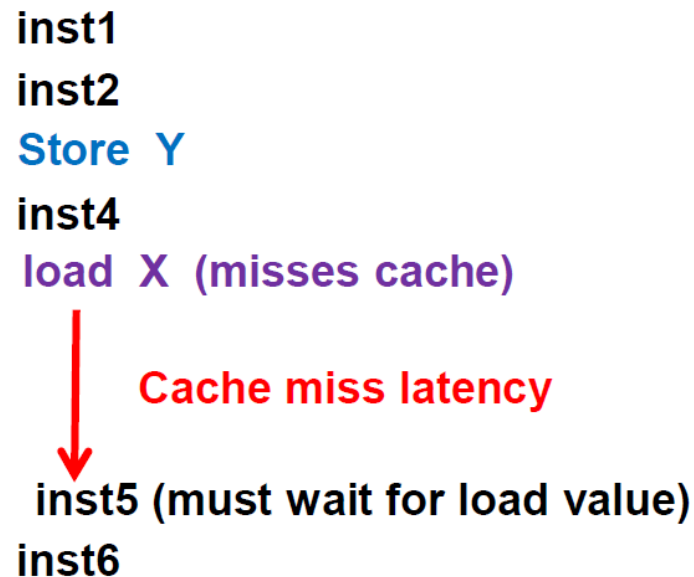
Background – Out-Of-Order Execution

Hide long latency of independent instructions (e.g., cache misses)



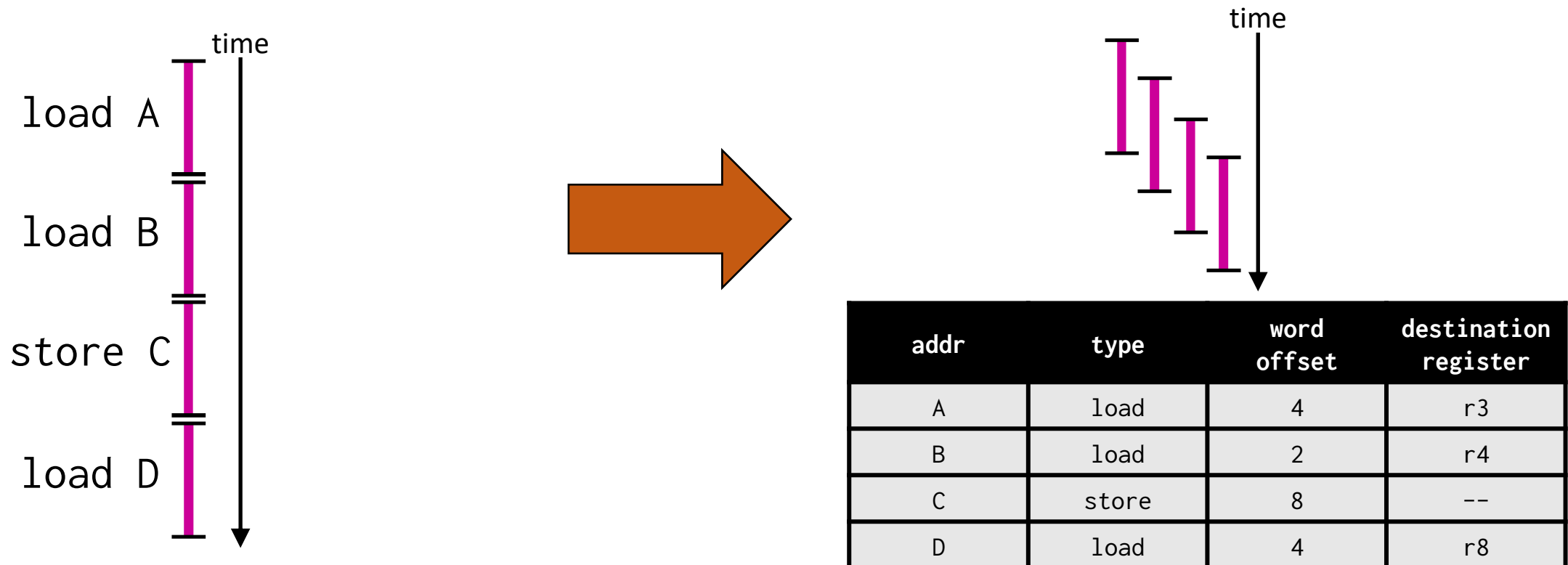
Background – Speculative Loads

Allow loads to execute before ambiguous stores and use load queue to verify later when the store address is eventually known



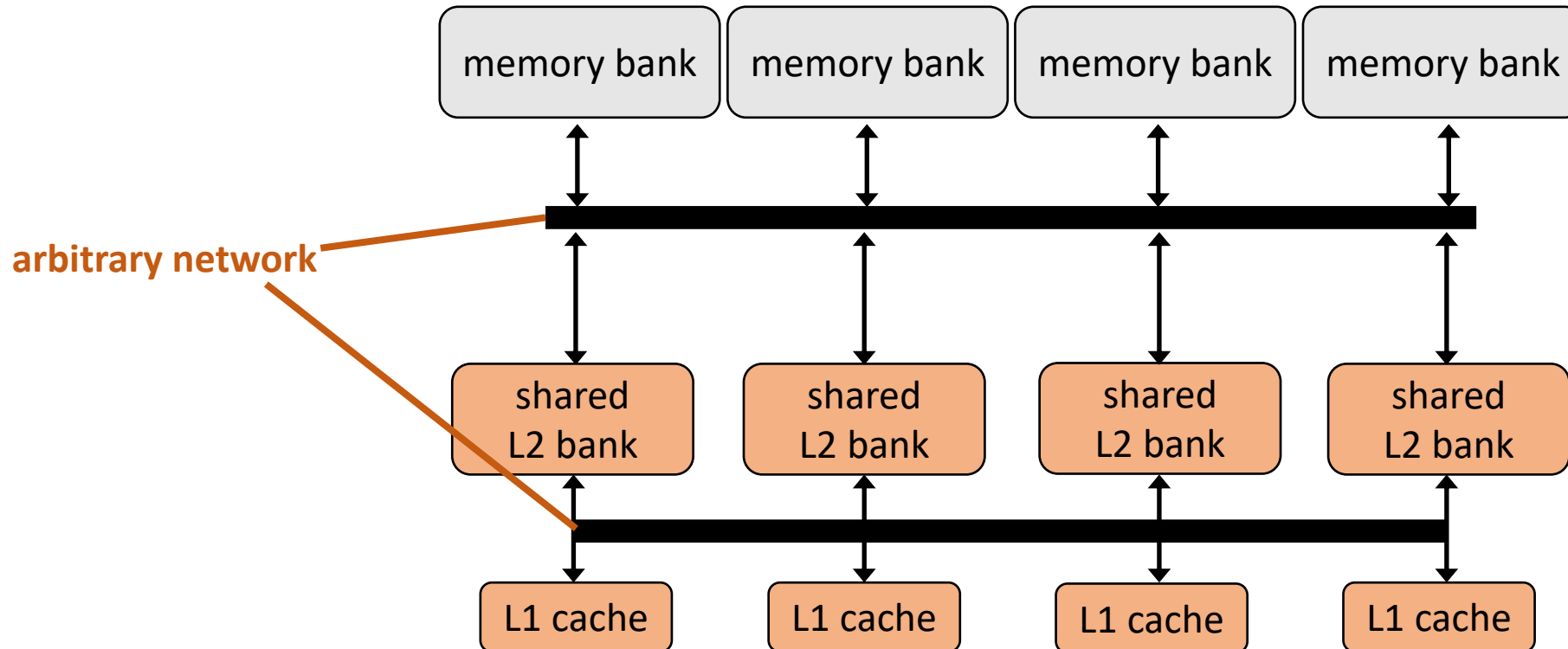
Background – Miss Status Handling Registers

With non-blocking cache, allow for multiple in-flight cache misses by tracking their state in miss status handling registers (MSHRs)



Background – NUCA and NUMA

Improve scalability by distributing cache and memory banks, resulting in non-uniform access latencies



Background – Value Prediction

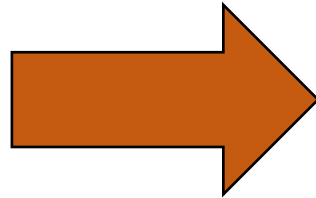
Predict values of load instructions to allow other instructions to execute while waiting to receive actual data value from memory

inst 1;
load X;

GetS X

Data X

inst 2;



inst 1;
load X;

inst 2;

inst 3;

inst 4;

inst 5;

verify X;

GetS X

Data X



Sequential Consistency

Fun Fact:

*Each of these optimizations (or combinations of them)
can violate sequential consistency!*



Sequential Consistency

“A multiprocessor is sequentially consistent (SC) if the result of any execution is the same as if the operations of all the processors were executed in some sequential order, and the operations of each individual processor appear in this sequence in the order specified by its program.”
[L. Lamport, IEEE Transactions on Computers 1979]

Memory is the main shared resource among threads in a program

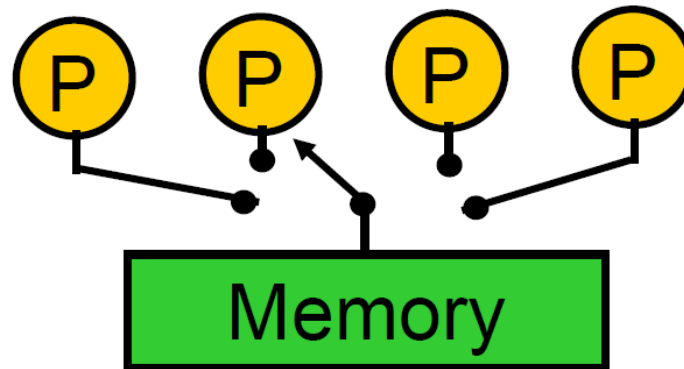
Sequential Consistency – Definition

To achieve sequential consistency, it should appear as if:

1. Each individual thread's reads and writes are performed in program order.



2. All memory reads/writes from all threads are performed globally serially.



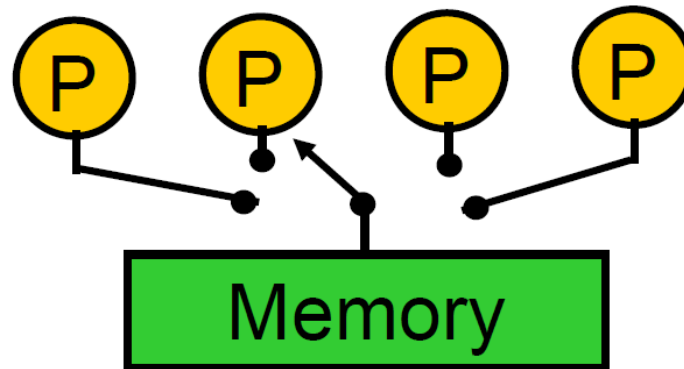
Sequential Consistency – Definition

To achieve sequential consistency, it should appear as if:

1. Each individual thread's reads and writes are **performed** in program order.



2. All memory reads/writes from all threads are **performed** globally serially.



Sequential Consistency – Definition

Defining **perform** [Scheurich and Dubois, ISCA 1987]:

- A load by core P_i is **performed with respect to** P_j when new stores to same address by P_j cannot affect the value returned by this load.
- A store by core P_i is **performed with respect to** P_j when a load by P_j from the same address returns the value defined by this (or some subsequent) store.

Sequential Consistency – Definition

Defining **perform** [Scheurich and Dubois, ISCA 1987]:

- A load by core P_i is **performed with respect to** P_j when new stores to same address by P_j cannot affect the value returned by this load.
- A store by core P_i is **performed with respect to** P_j when a load by P_j from the same address returns the value defined by this (or some subsequent) store.
- A store is **globally performed** if it is performed with respect to all cores.
- A load is **globally performed** if it is performed with respect to all cores and if the store that is the source of this load's returned value has been globally performed.

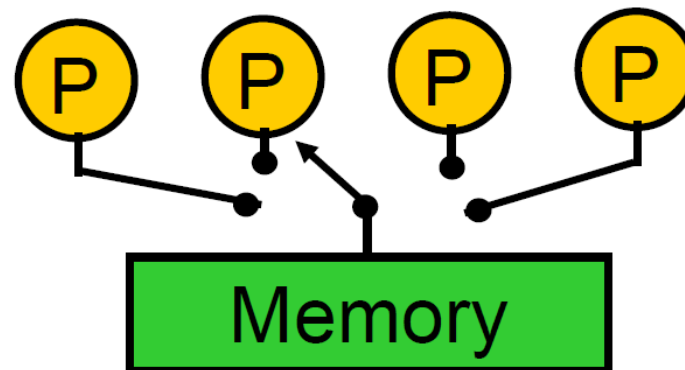
Sequential Consistency – Definition

To achieve sequential consistency, it should appear as if:

1. Reads/writes of each individual core are globally performed in program order.



2. A read/write may not be performed with respect to any core until the previous read/write has been globally performed.



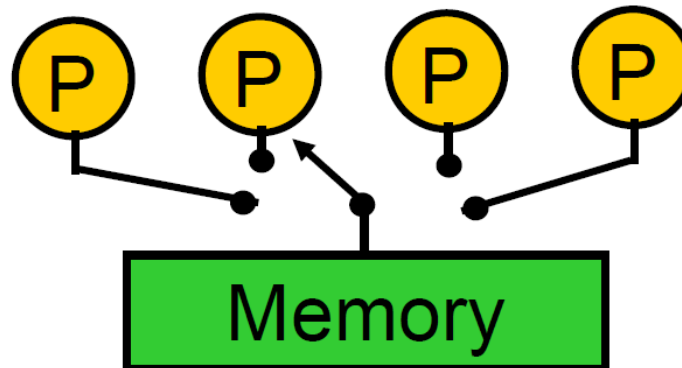
Sequential Consistency – Definition

To achieve sequential consistency, it should **appear** as if:

1. Reads/writes of each individual core are globally performed in program order.



2. A read/write may not be performed with respect to any core until the previous read/write has been globally performed.



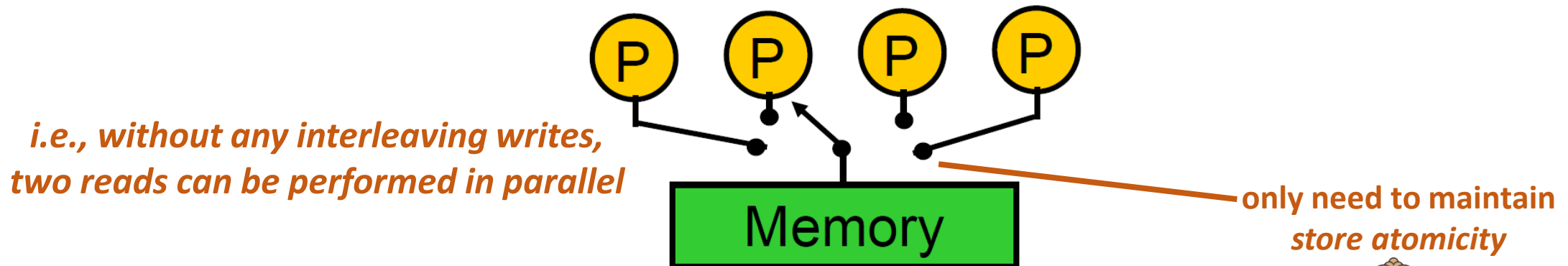
Sequential Consistency – Definition

To achieve sequential consistency, it should **appear** as if:

1. Reads/writes of each individual core are globally performed in program order.



2. A read/write may not be performed with respect to any core until the previous read/write has been globally performed.



Sequential Consistency – Implementation

Requirements for a sequentially consistent processor:

Program Ordering:

Each individual core's reads and writes must be globally performed in program order.

Store Atomicity:

If a write is performed with respect to core P_i before another write is performed with respect to P_i , the two writes must be performed in the same order with respect to any other core P_j .

- Property that states the existence of a total order of all changes to memory state (i.e., writes)

Sequential Consistency – Implementation

In other words...

Program Ordering:

Each core/thread must make sure its own reads and writes are globally performed in the order specified by the programmer.

Store Atomicity:

All cores/threads must agree on the order that writes are performed with respect to them.

Consistency and Coherence

Coherence concerns ordering of reads and writes to **one** memory location.

Consistency concerns ordering of reads and writes to **all** memory locations.

Consistency and Coherence

Coherence concerns ordering of reads and writes to **one** memory location.

Consistency concerns ordering of reads and writes to **all** memory locations.

*Coherence is necessary for sequential consistency,
but providing coherence alone is not sufficient*

Reasoning About SC

Construct a directed constraint graph:

- Nodes: Dynamic instances of memory read/write
- Edges: Transitive ordering (program order, RAW, WAR, WAW)

Reasoning About SC

Construct a directed constraint graph:

- Nodes: Dynamic instances of memory read/write
- Edges: Transitive ordering (program order, RAW, WAR, WAW)

If the constraint graph is acyclic, then the execution is sequentially consistent

Reasoning About SC

// Possible: A=0, B=1?

*x = 0; *y = 0;

// P1

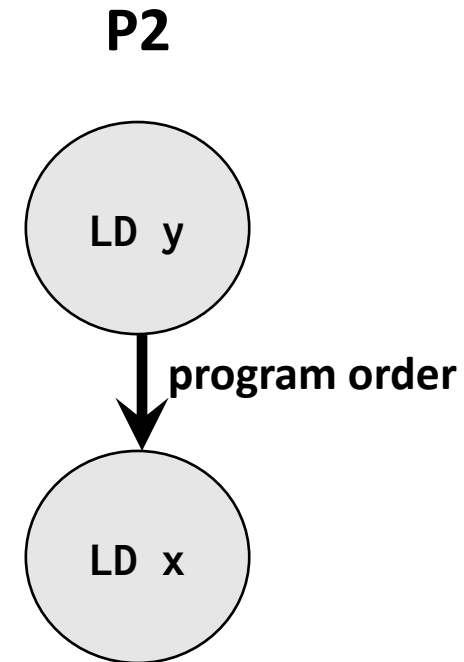
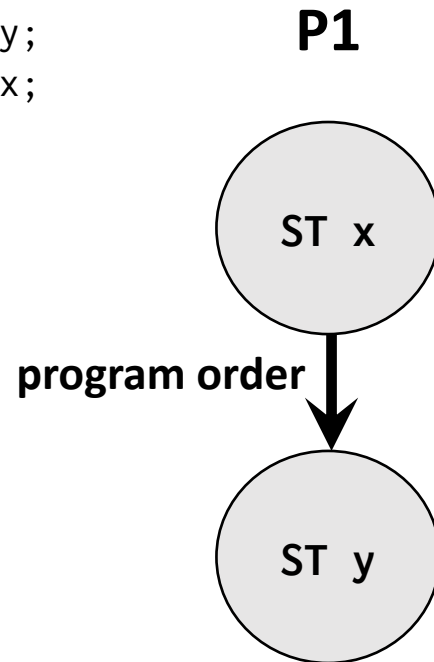
*x = 1;

*y = 1;

// P2

A = *y;

B = *x;



Reasoning About SC

// Possible: A=0, B=1?

*x = 0; *y = 0;

// P1

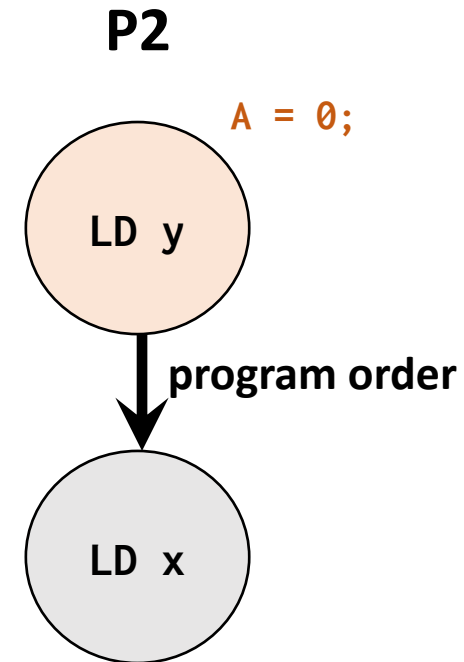
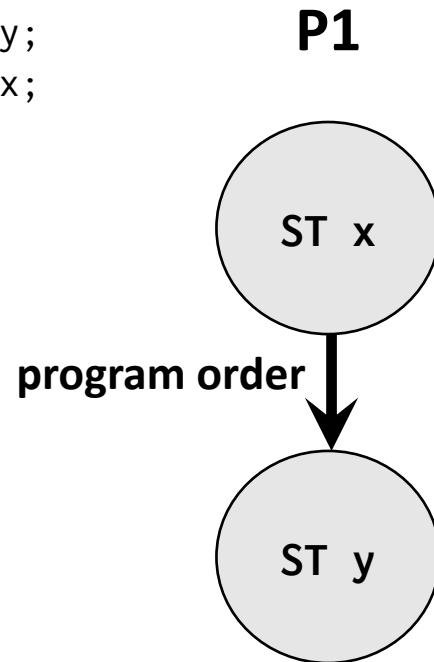
*x = 1;

*y = 1;

// P2

A = *y;

B = *x;



Reasoning About SC

// Possible: A=0, B=1?

*x = 0; *y = 0;

// P1

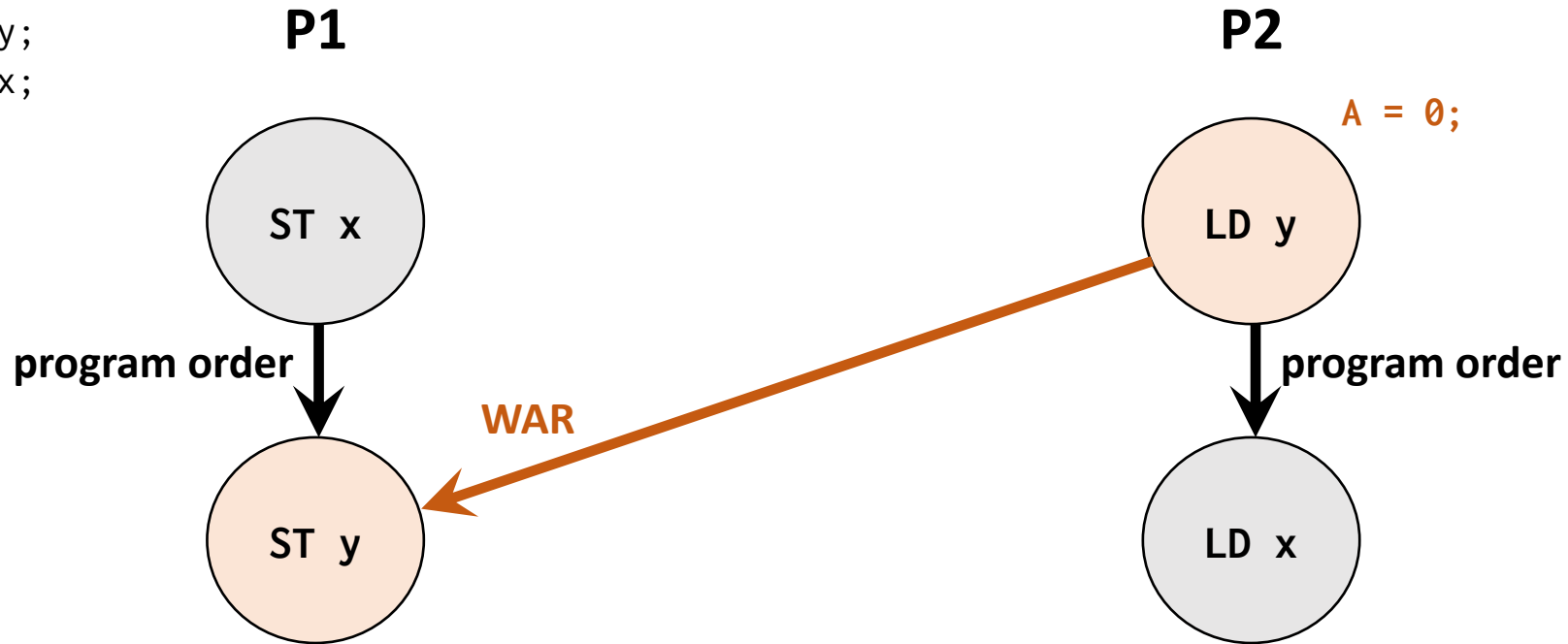
*x = 1;

*y = 1;

// P2

A = *y;

B = *x;



Reasoning About SC

// Possible: A=0, B=1?

*x = 0; *y = 0;

// P1

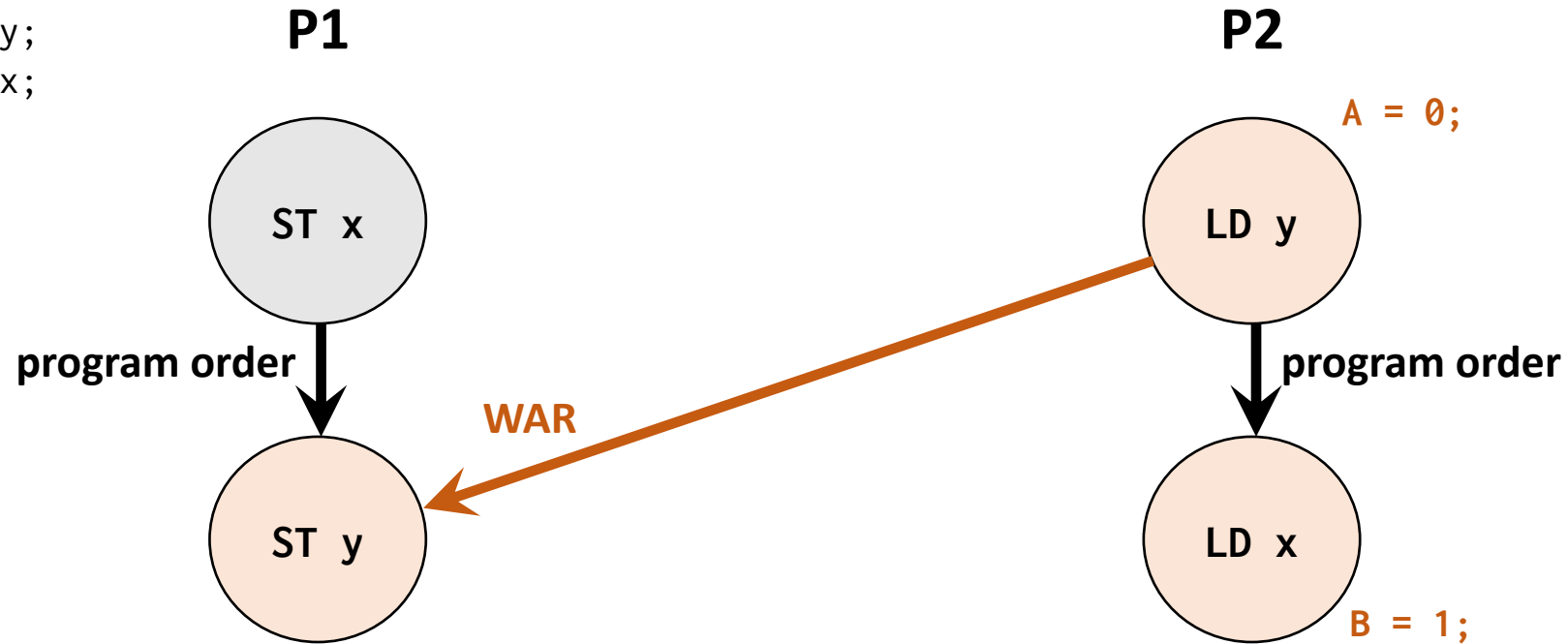
*x = 1;

*y = 1;

// P2

A = *y;

B = *x;



Reasoning About SC

// Possible: A=0, B=1?

*x = 0; *y = 0;

// P1

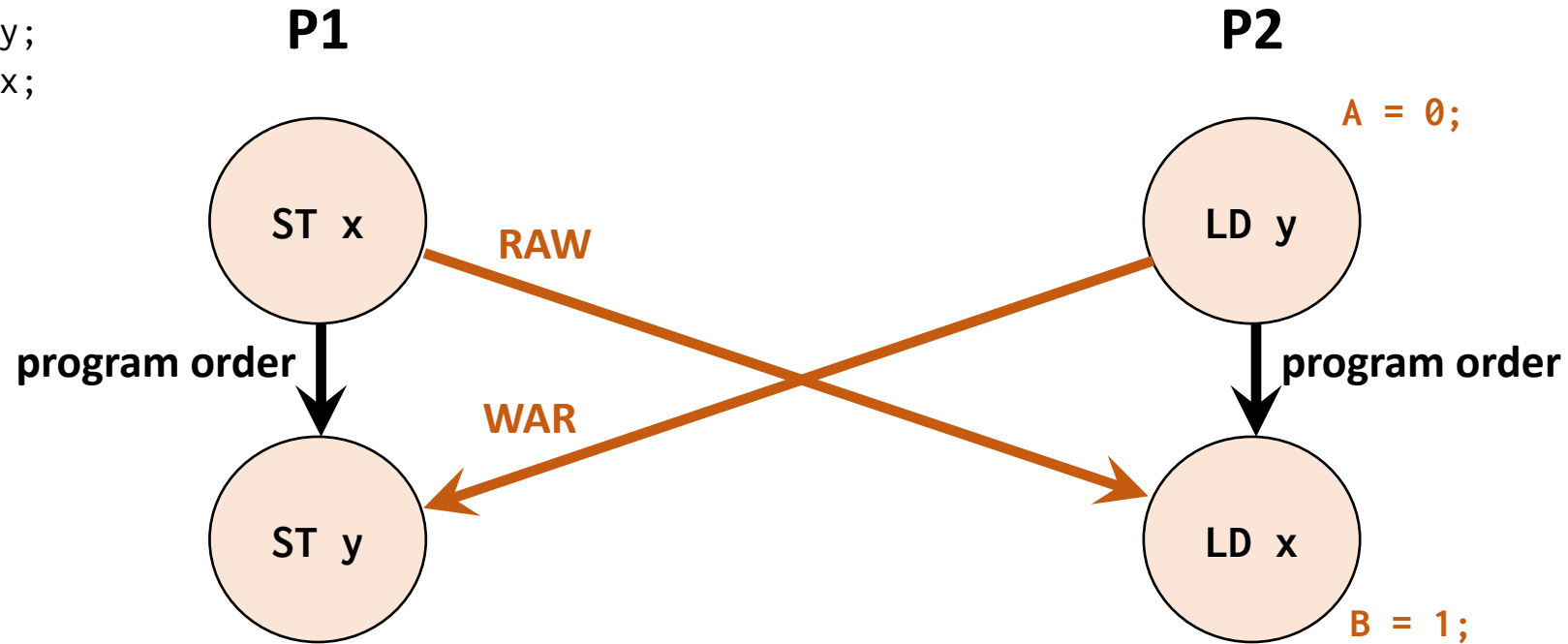
*x = 1;

*y = 1;

// P2

A = *y;

B = *x;



Reasoning About SC

// Possible: A=0, B=1? **Yes**

*x = 0; *y = 0;

// P1

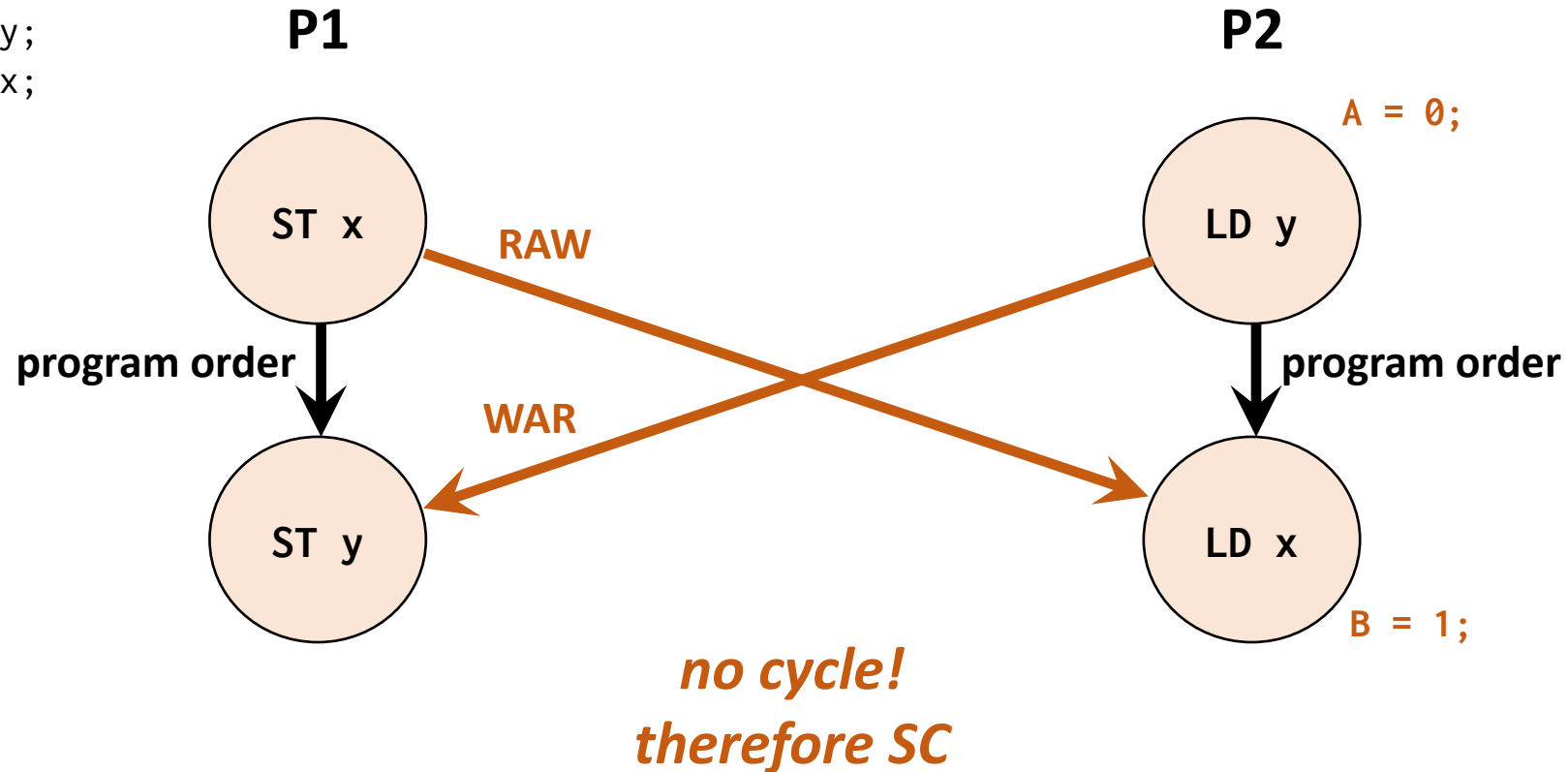
*x = 1;

*y = 1;

// P2

A = *y;

B = *x;



Reasoning About SC

// Possible: A=1, B=0?

*x = 0; *y = 0;

// P1

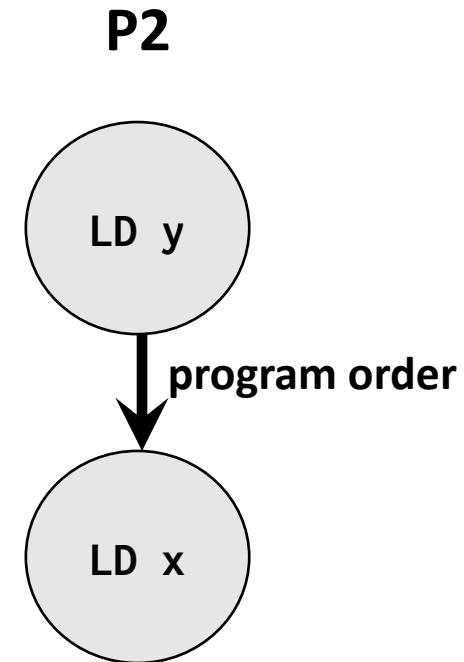
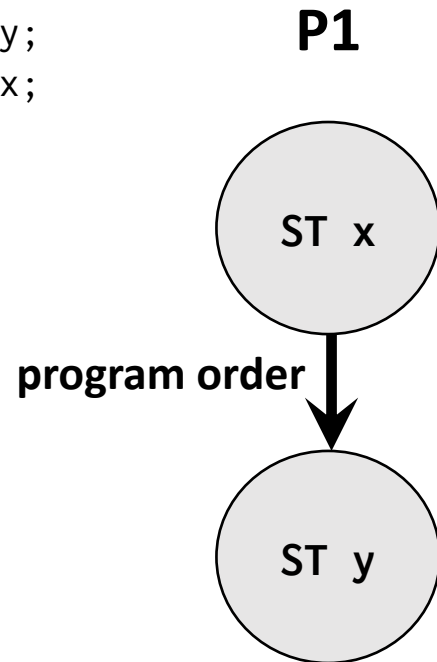
*x = 1;

*y = 1;

// P2

A = *y;

B = *x;



Reasoning About SC

// Possible: A=1, B=0?

*x = 0; *y = 0;

// P1

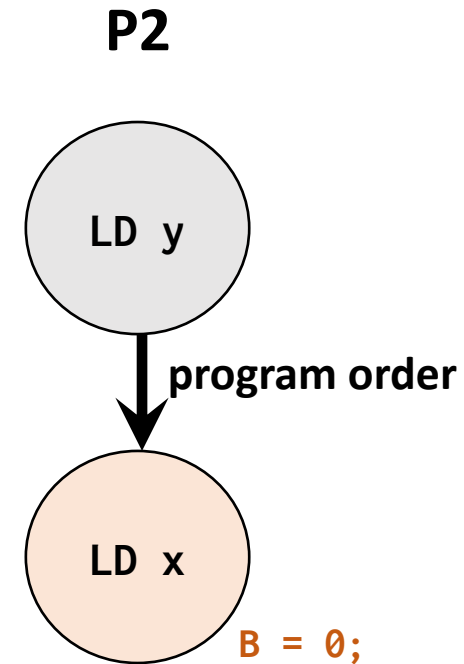
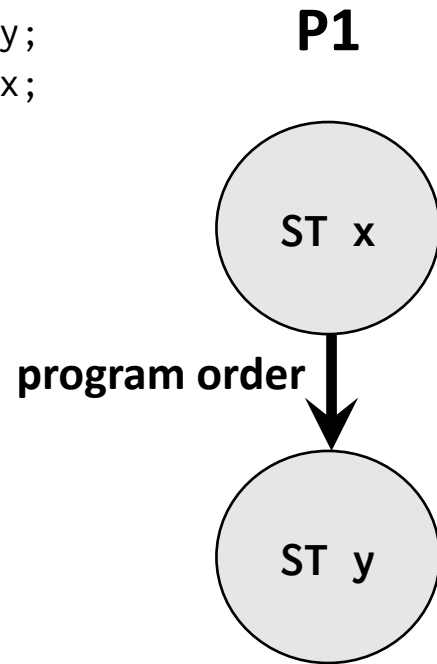
*x = 1;

*y = 1;

// P2

A = *y;

B = *x;



Reasoning About SC

// Possible: A=1, B=0?

*x = 0; *y = 0;

// P1

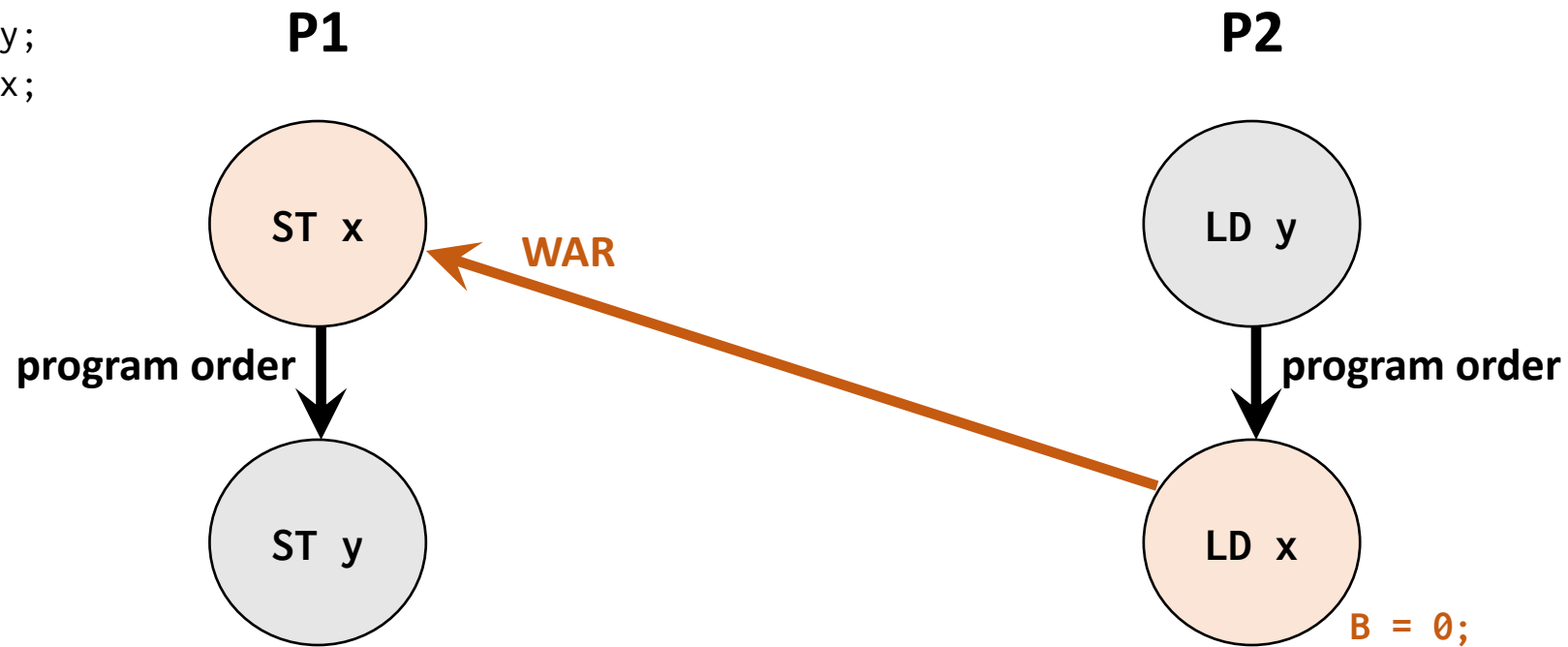
*x = 1;

*y = 1;

// P2

A = *y;

B = *x;



Reasoning About SC

// Possible: A=1, B=0?

*x = 0; *y = 0;

// P1

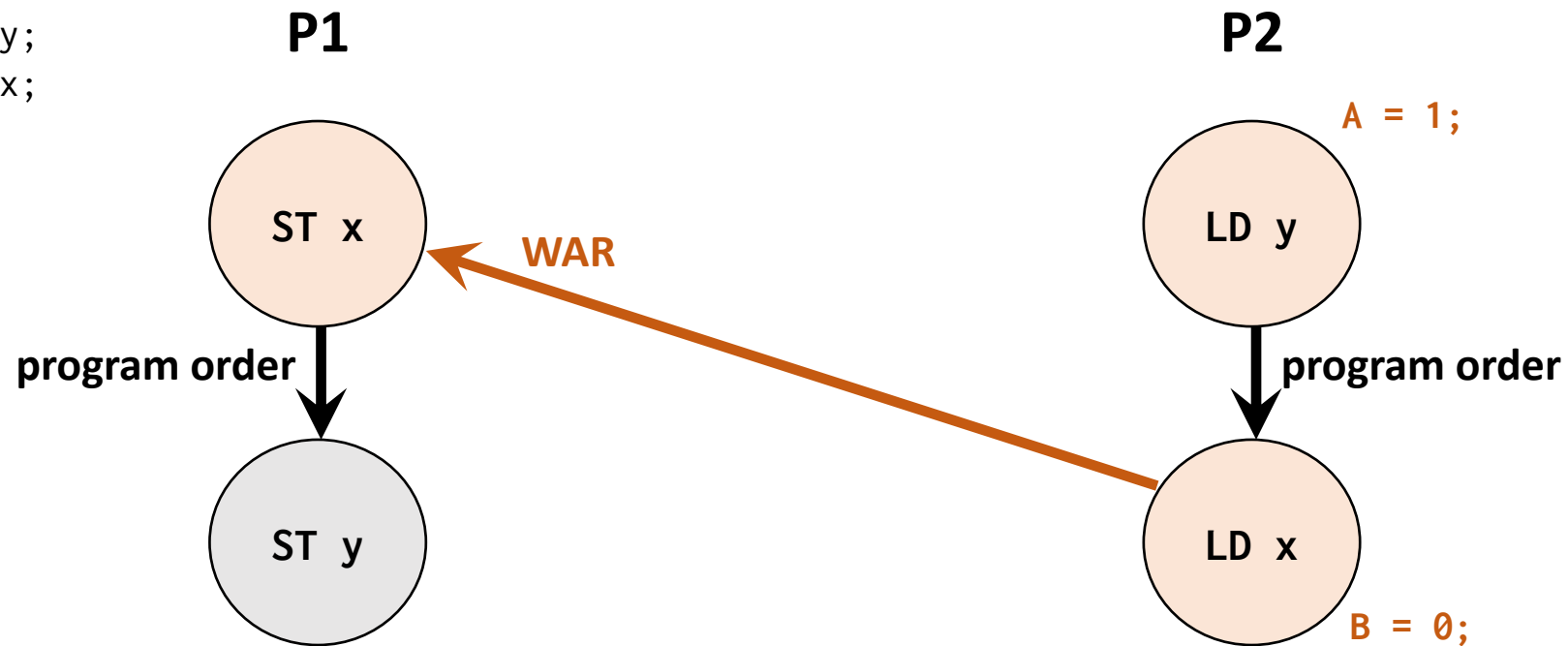
*x = 1;

*y = 1;

// P2

A = *y;

B = *x;



Reasoning About SC

// Possible: A=1, B=0?

*x = 0; *y = 0;

// P1

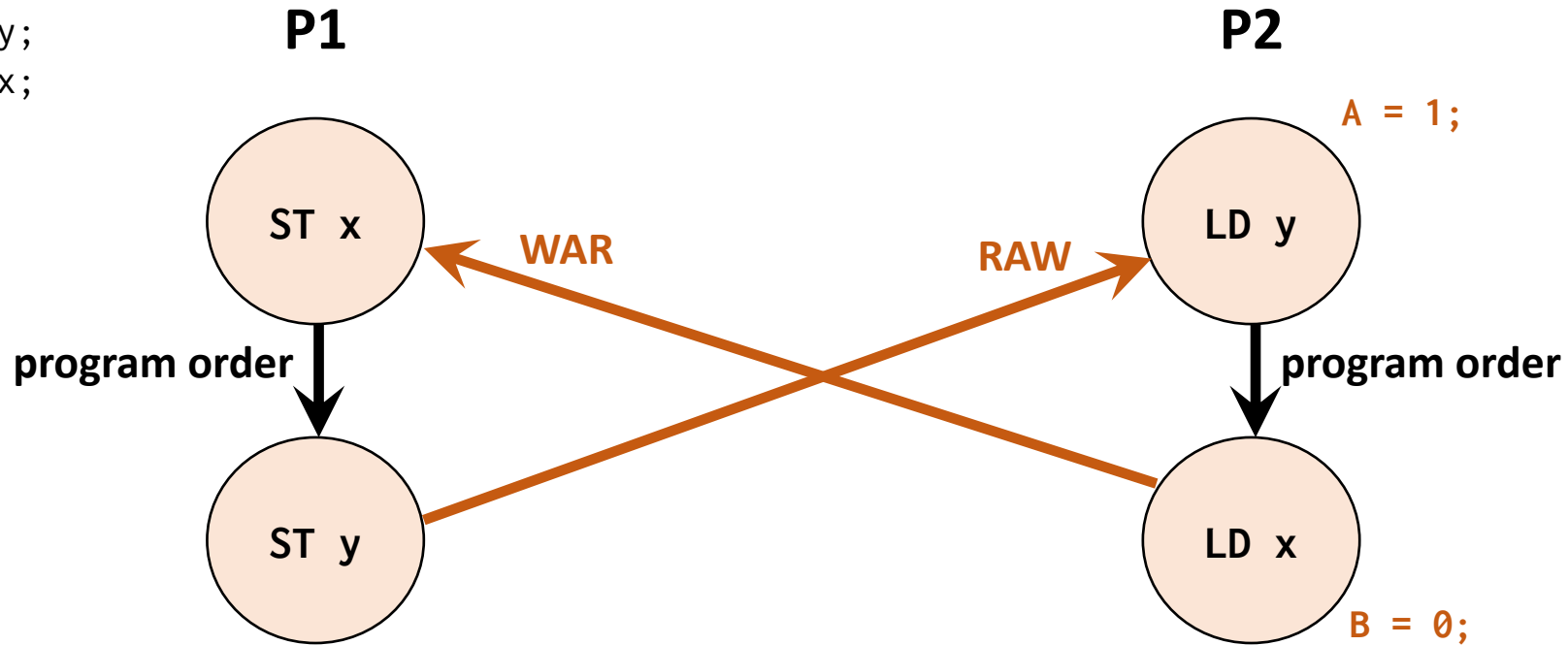
*x = 1;

*y = 1;

// P2

A = *y;

B = *x;



Reasoning About SC

// Possible: A=1, B=0? **No**

*x = 0; *y = 0;

// P1

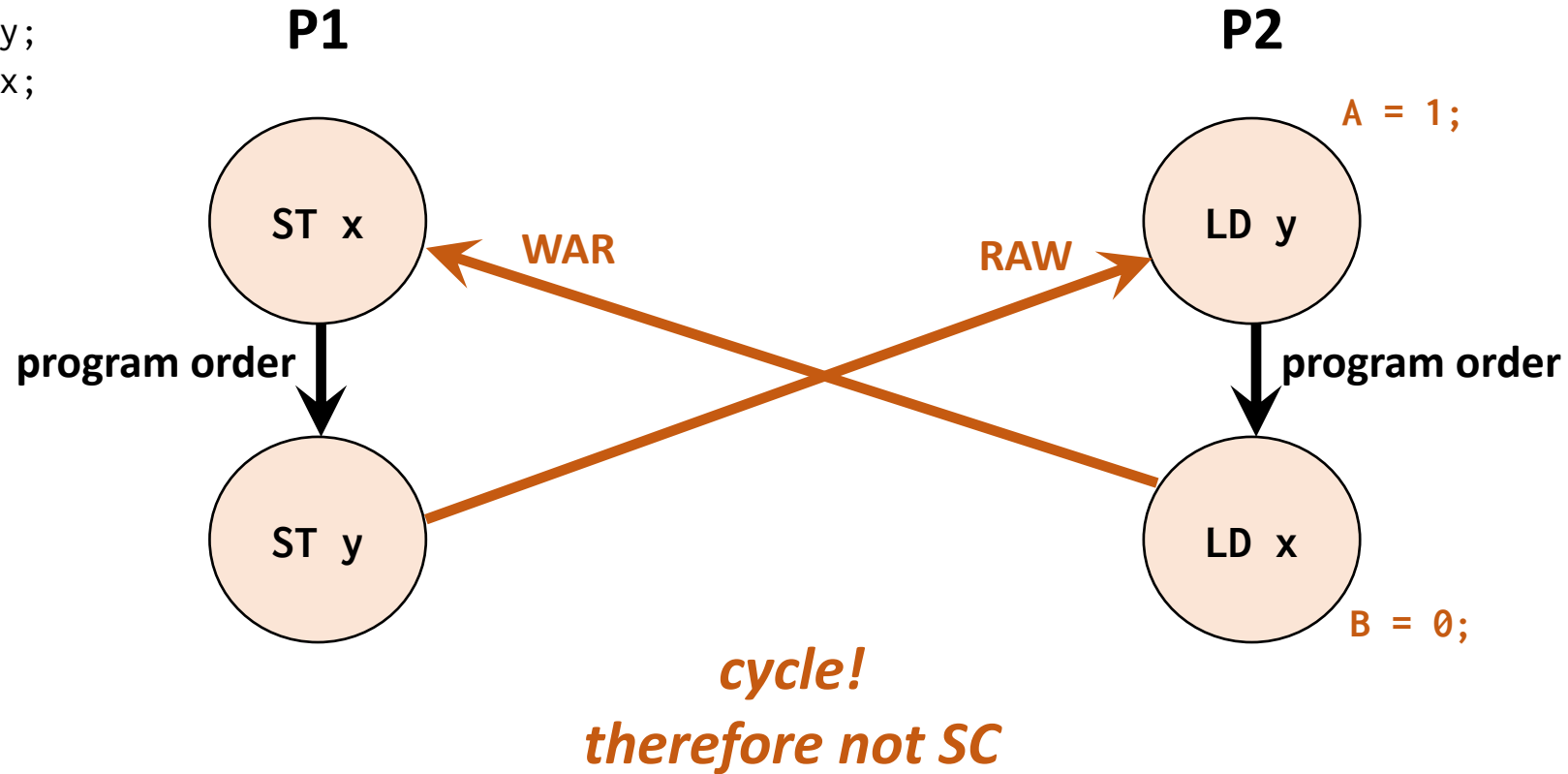
*x = 1;

*y = 1;

// P2

A = *y;

B = *x;



Causality

If I can observe the result of what you just did, I must also be able to observe the results of everything else you've done so far.

Causality

If I can observe the result of what you just did, I must also be able to observe the results of everything else you've done so far.

- No data dependences across multiple threads in the beginning
- Dependences are revealed during execution

Causality

If I can observe the result of what you just did, I must also be able to observe the results of everything else you've done so far.

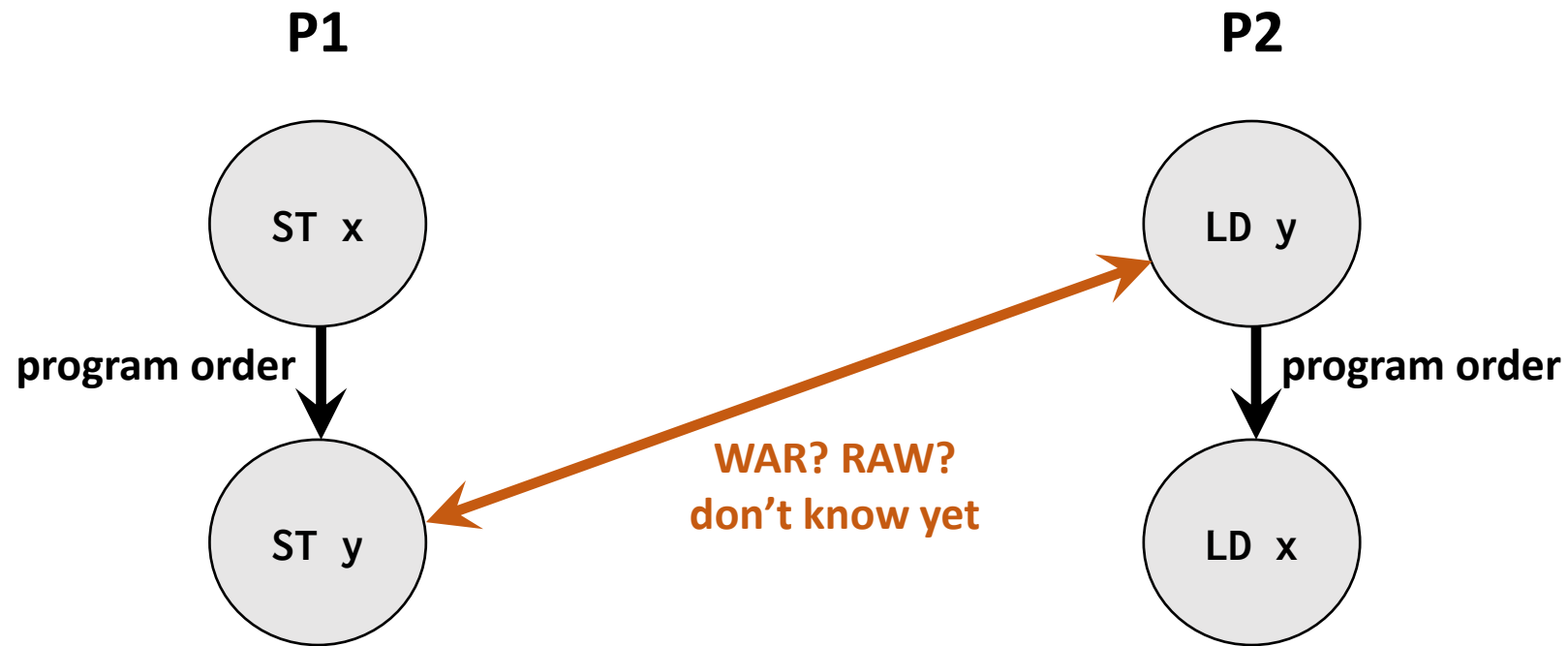
- No data dependences across multiple threads in the beginning
- Dependences are revealed during execution

Causality effectively provides the bound on which memory accesses you are (or are not) allowed to see

Causality



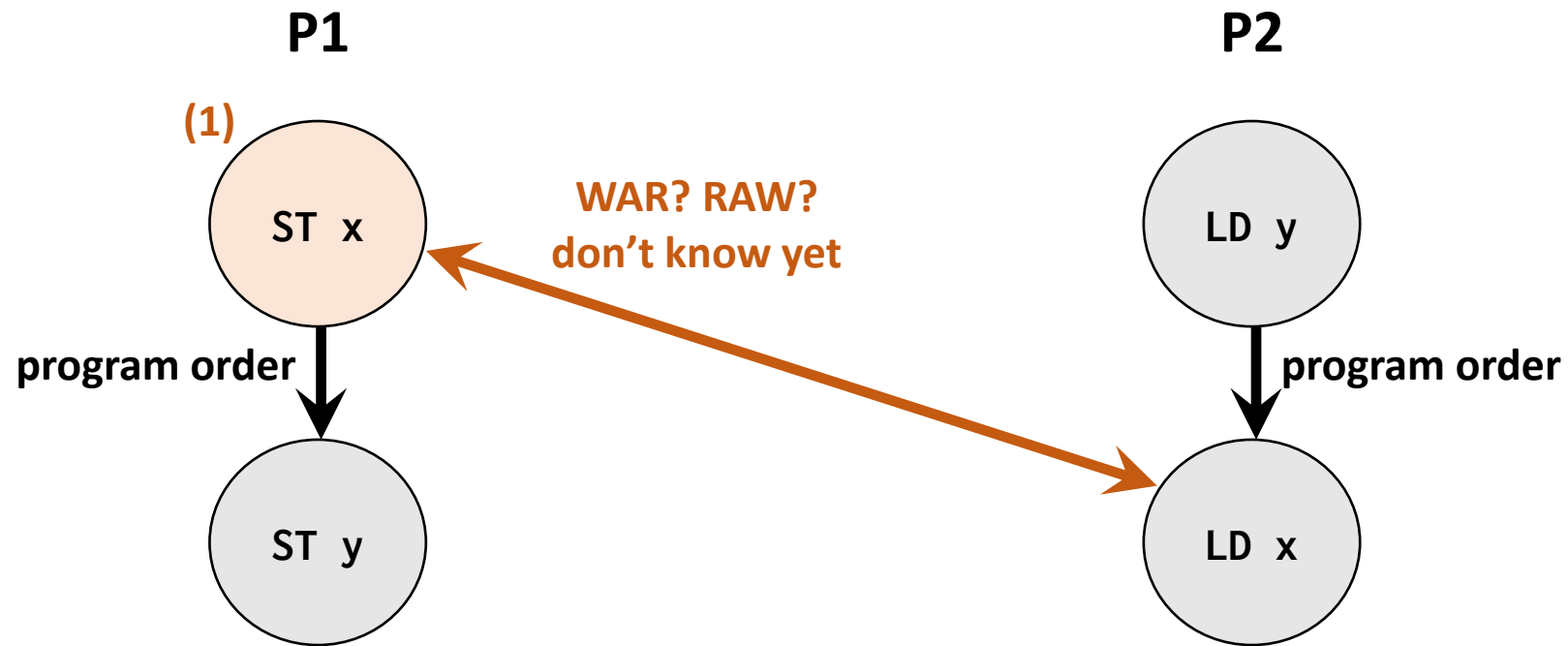
Causality



Causality



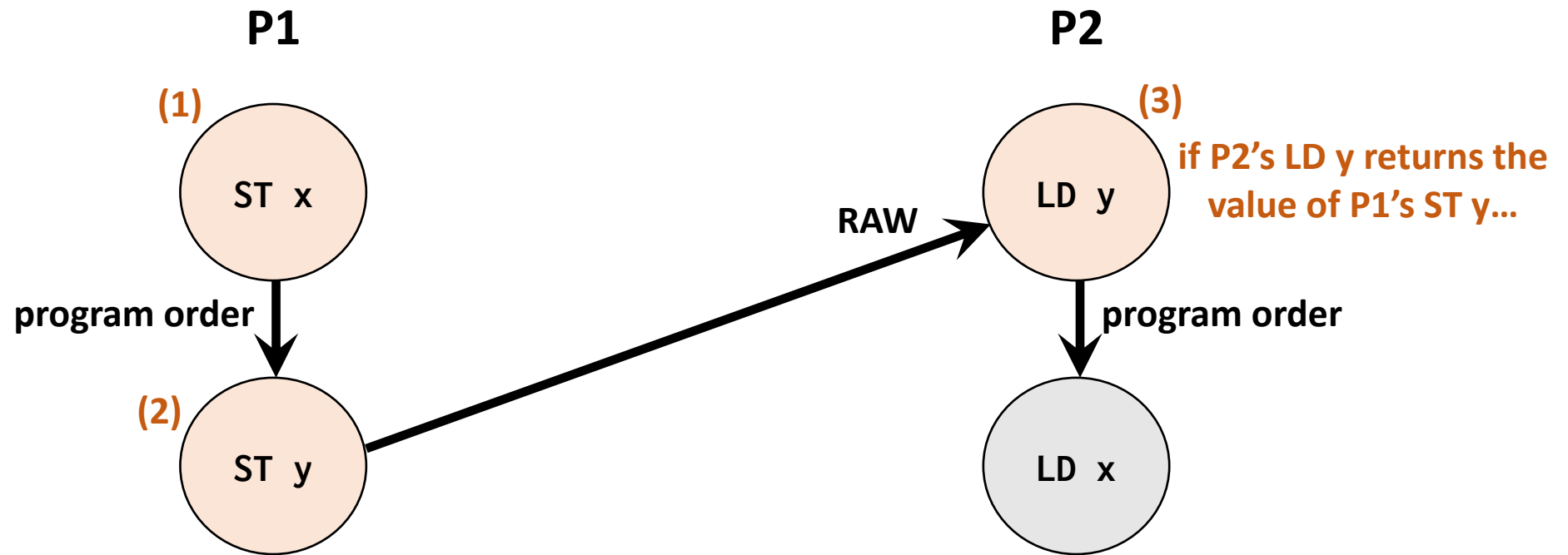
Causality



Causality

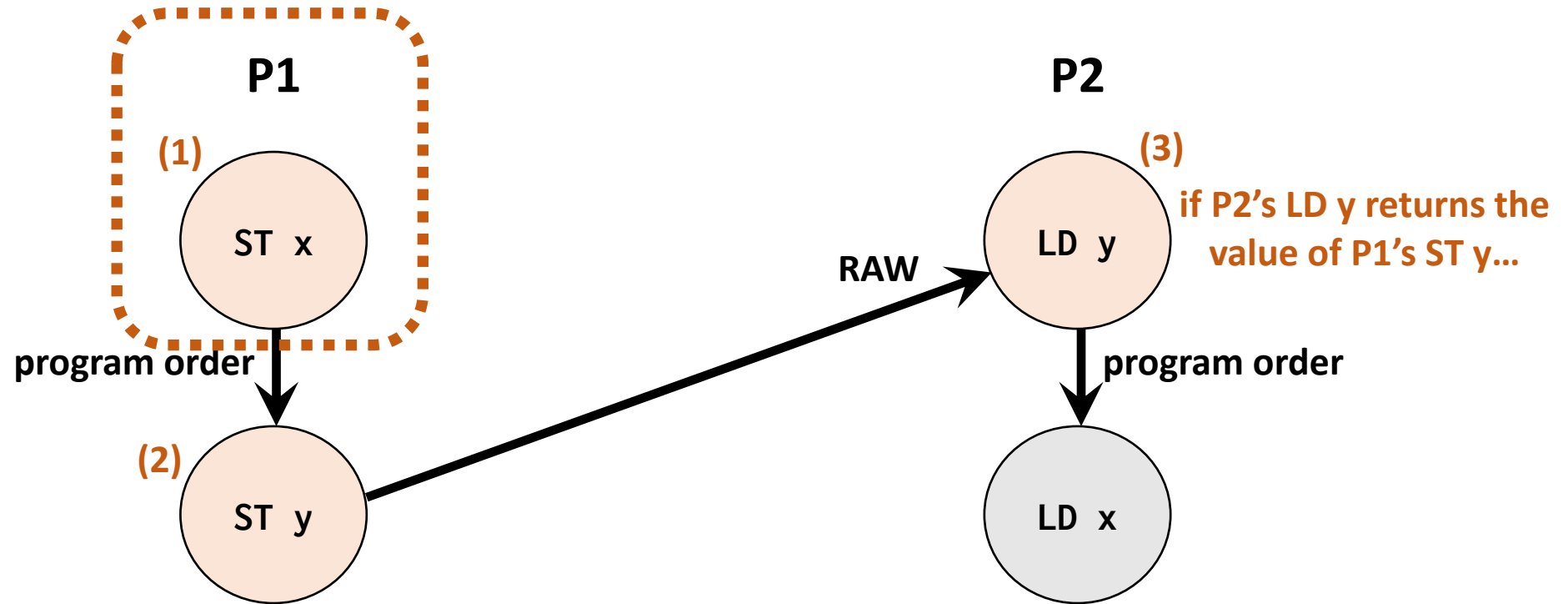


Causality



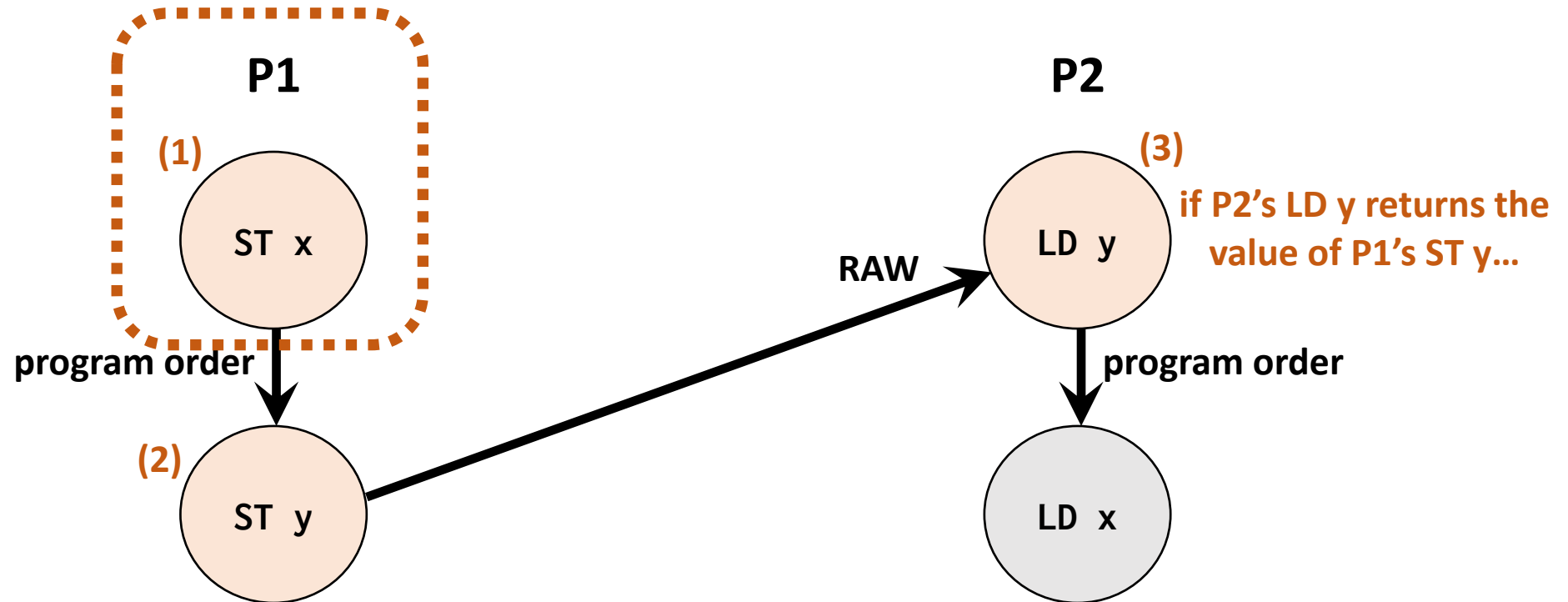
Causality

...causality states that P2 must now observe P1's earlier writes



Causality

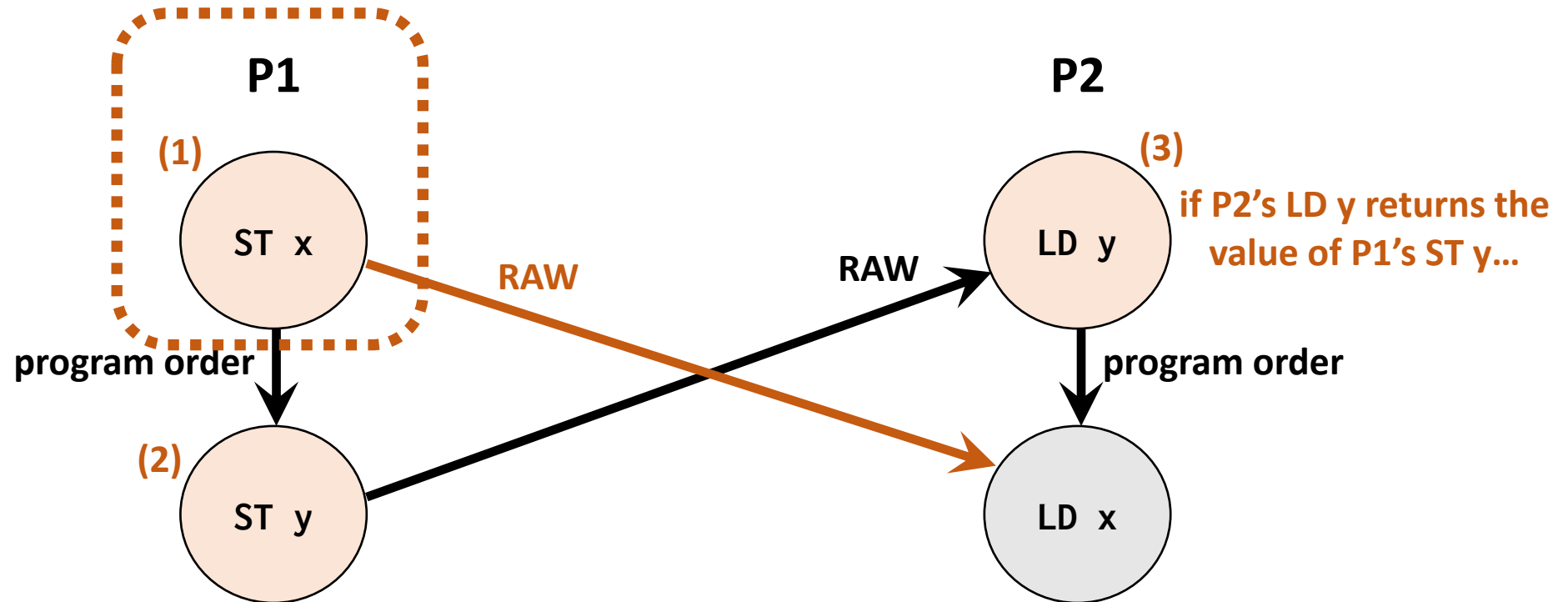
...causality states that P2 must now observe P1's earlier writes



in other words, because of this dependence with y,
LD x must now view ST x as the "past"

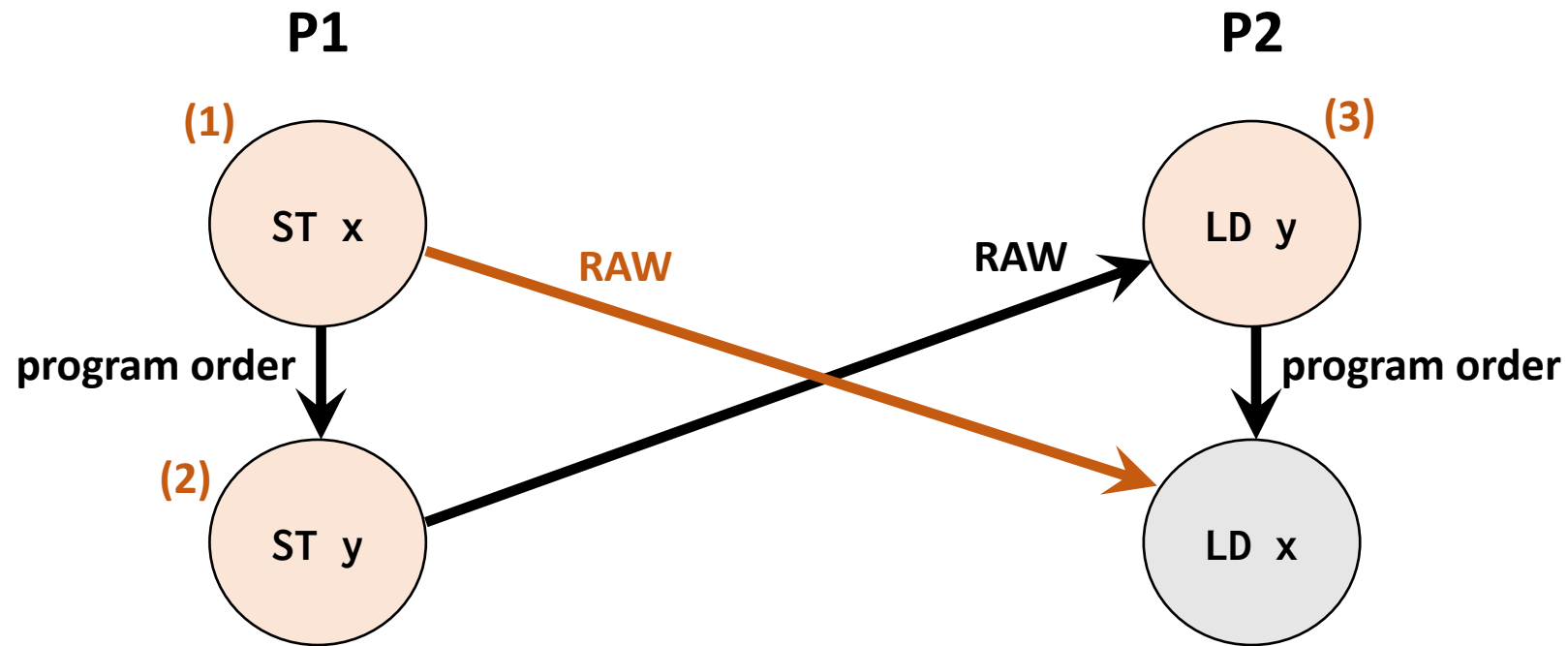
Causality

...causality states that P2 must now observe P1's earlier writes

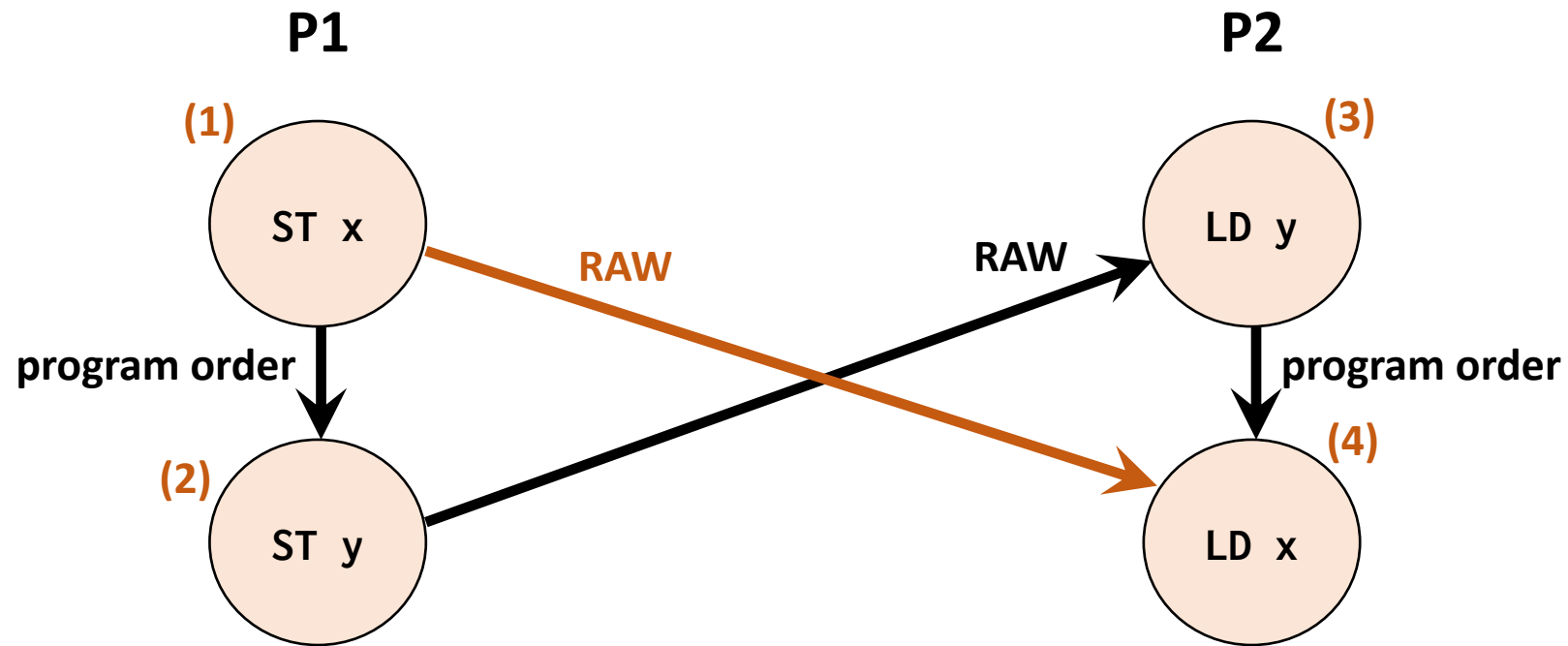


in other words, because of this dependence with y,
LD x must now view ST x as the "past"

Causality

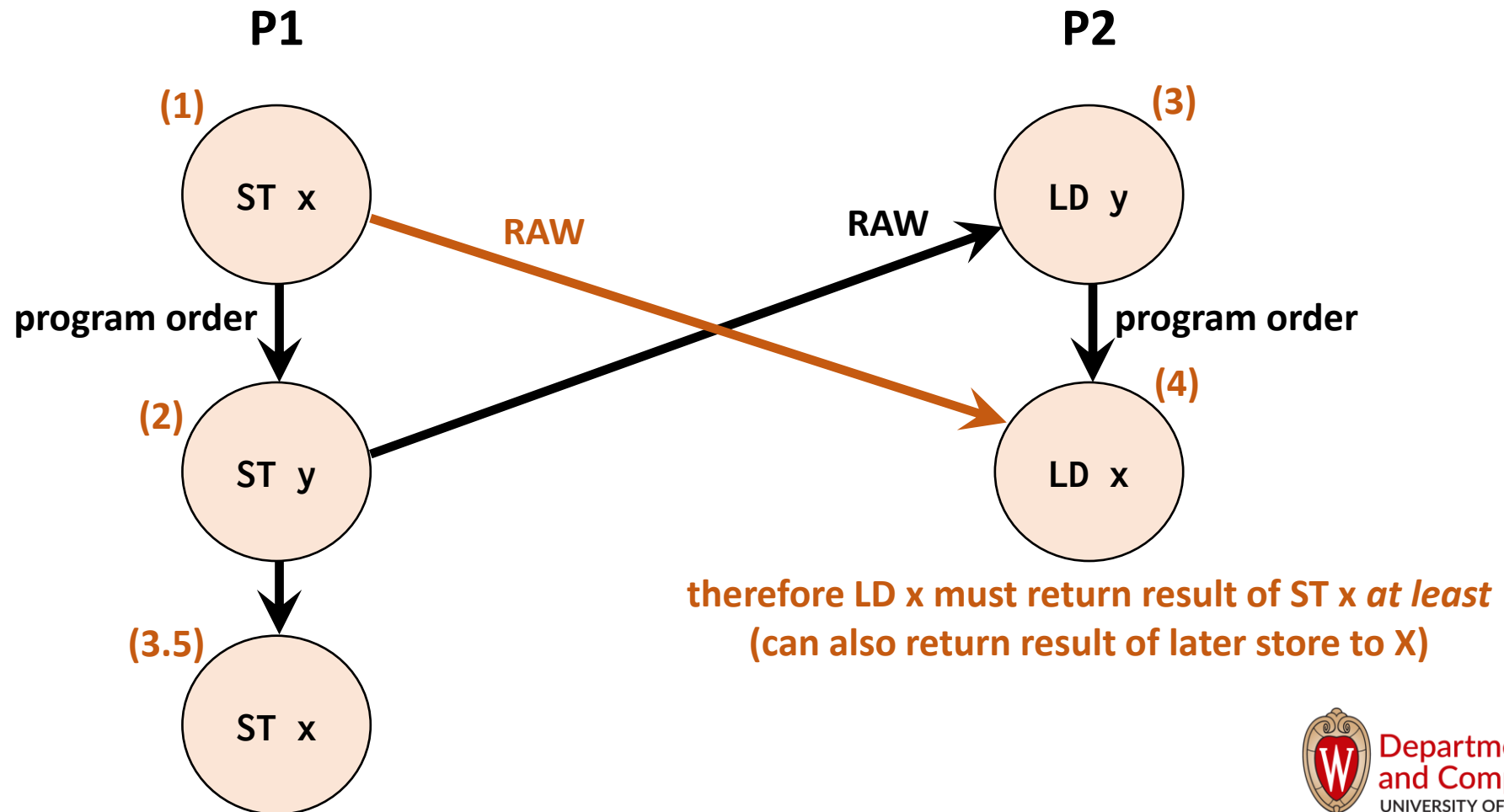


Causality

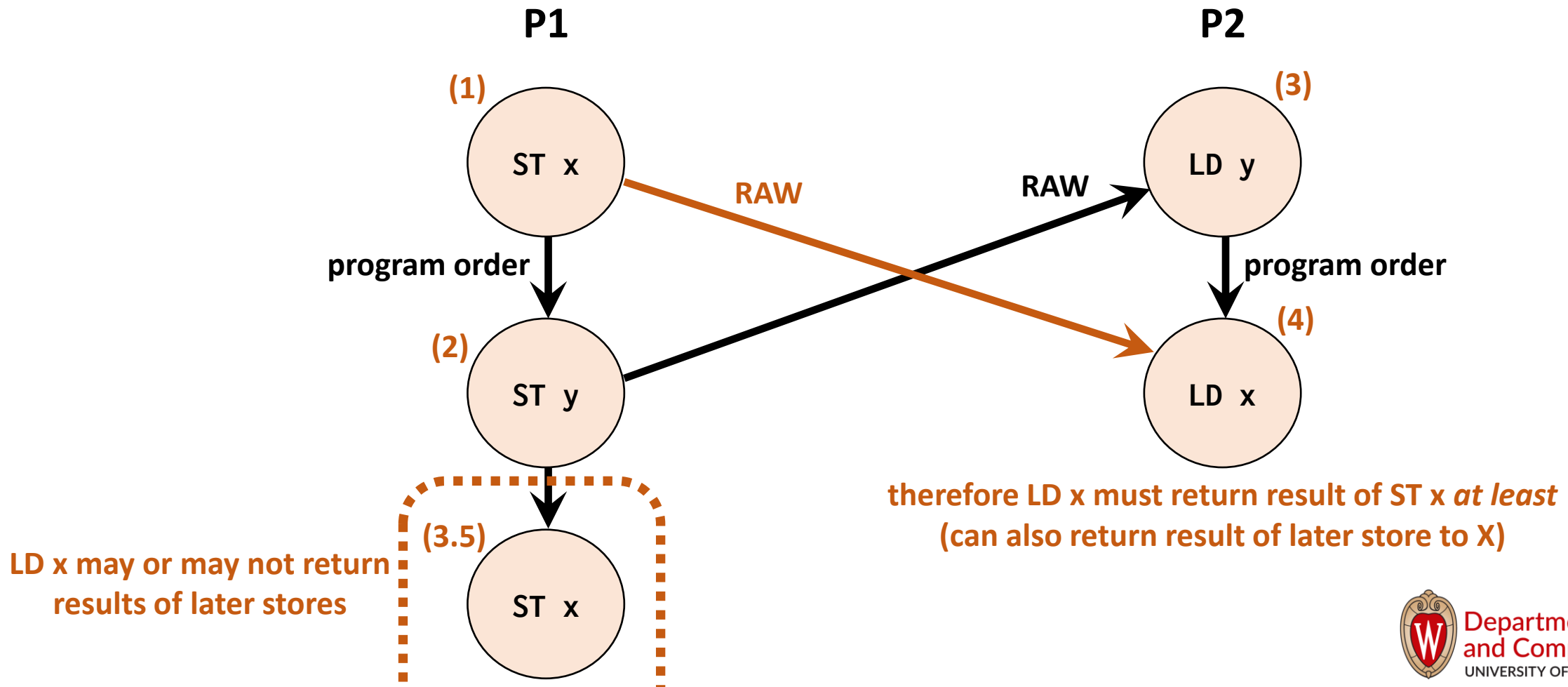


therefore LD x must return result of ST x *at least*

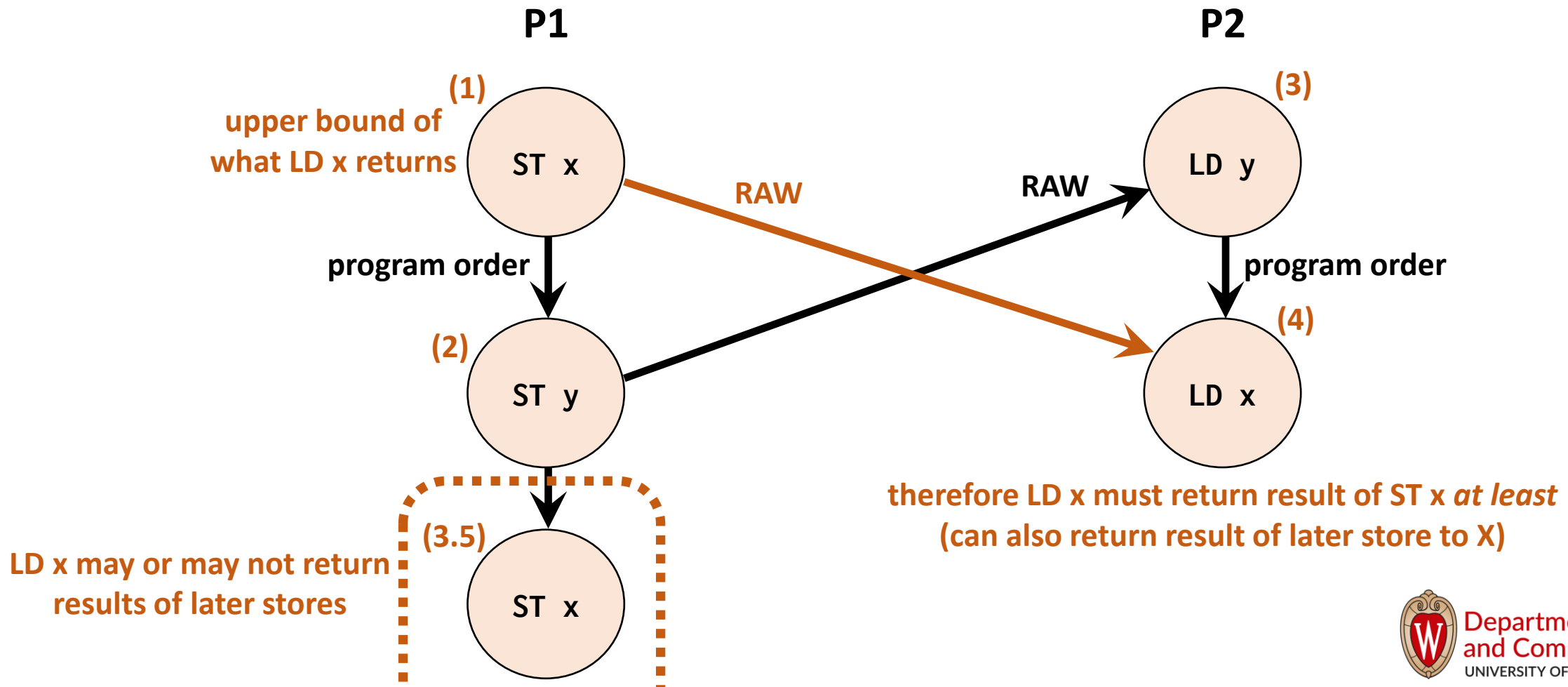
Causality



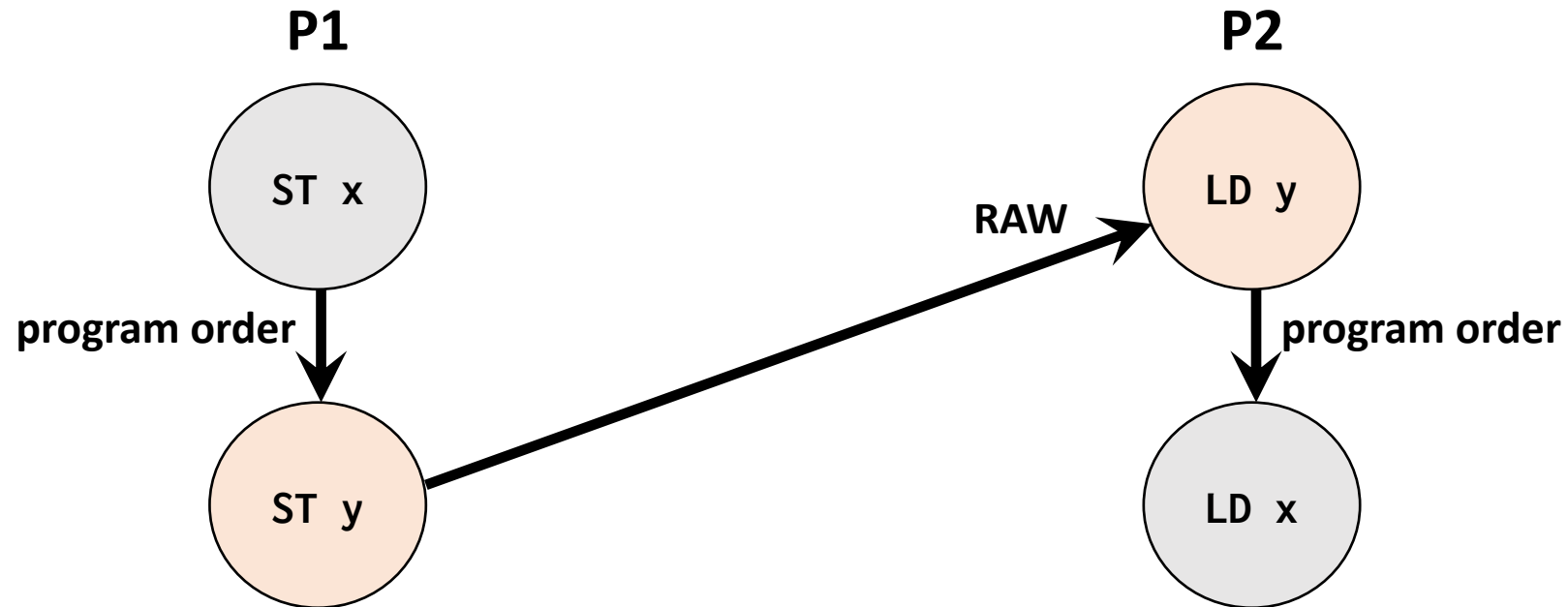
Causality



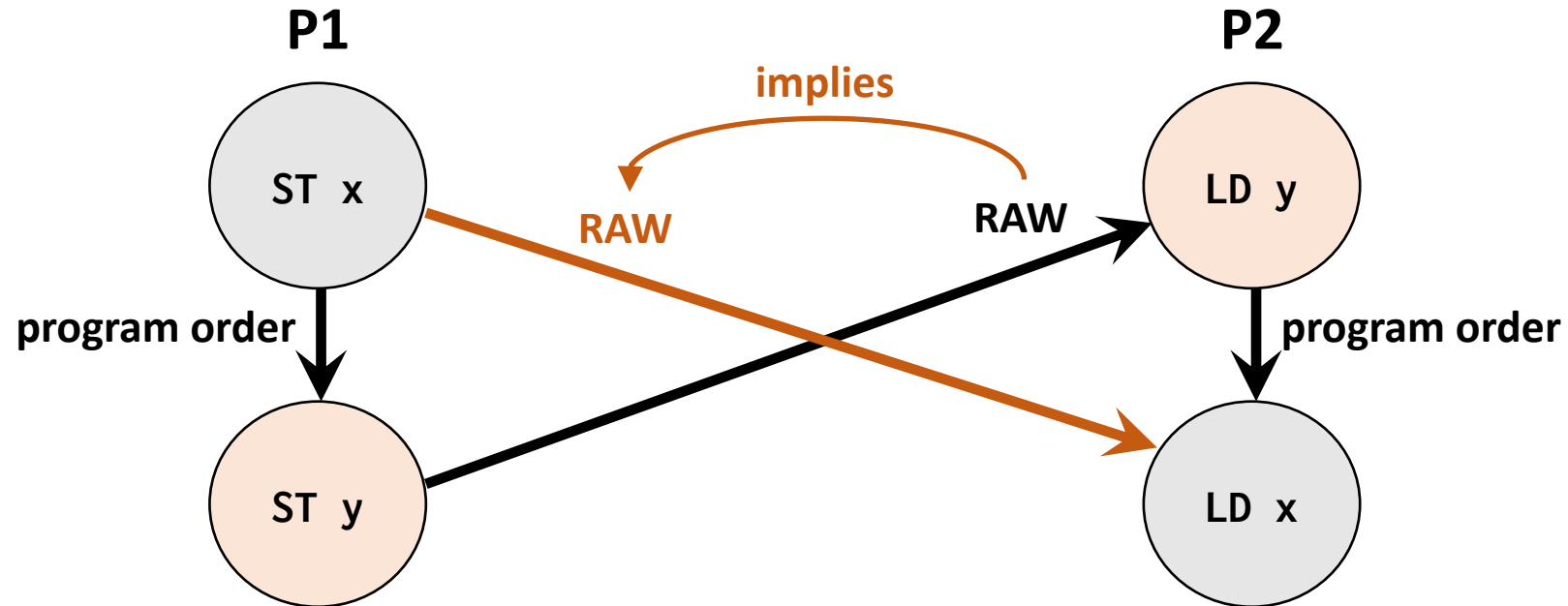
Causality



Causality

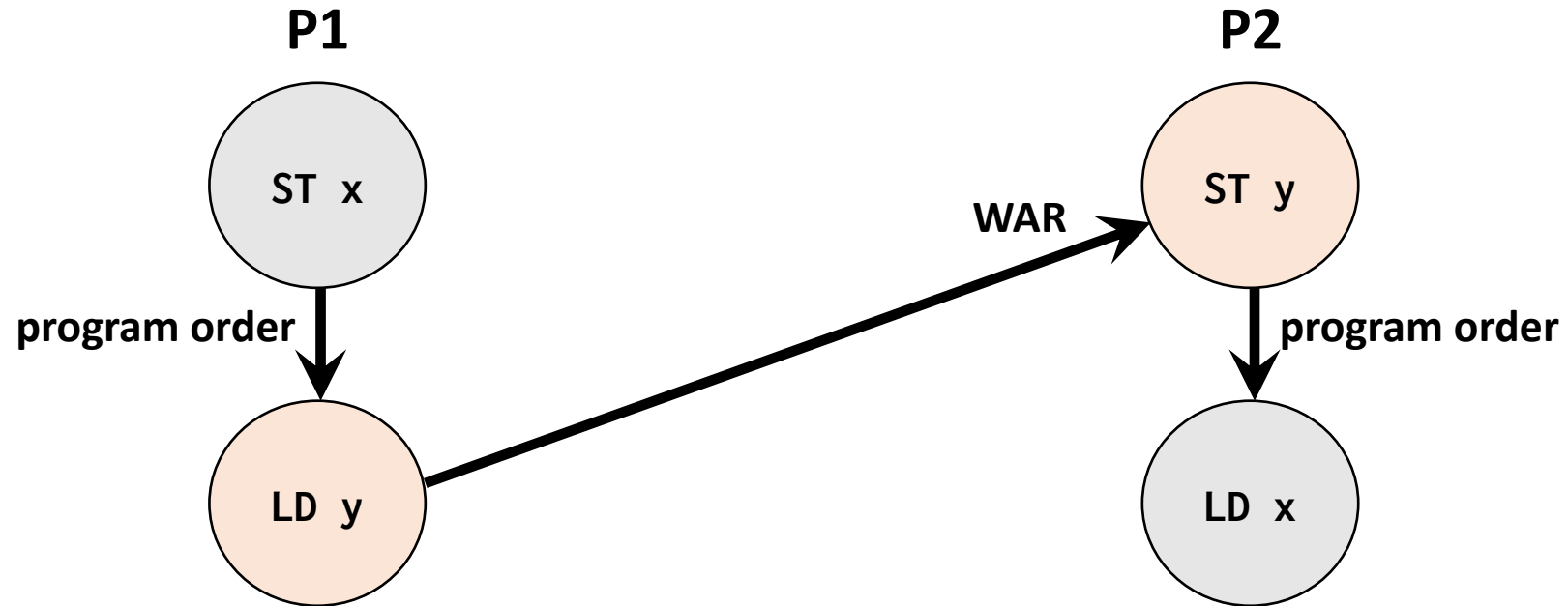


Causality

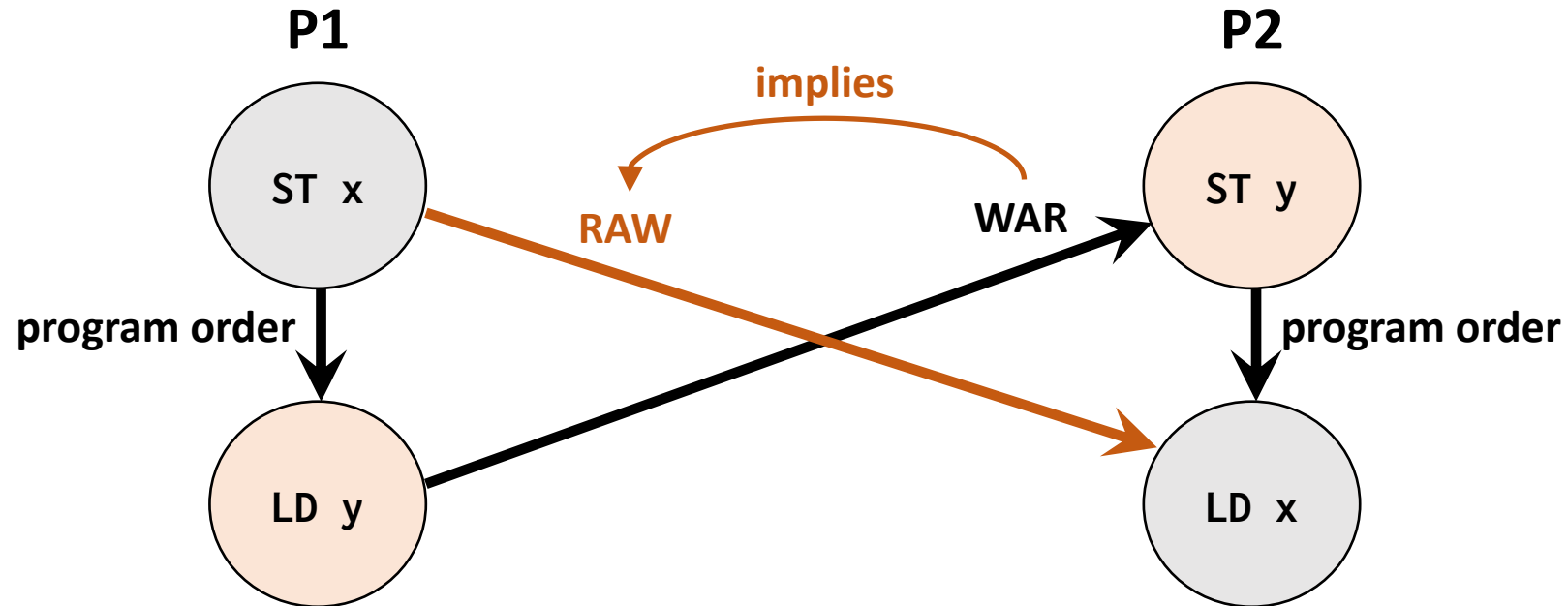


Causality implies that P2's LD x must get its value *at least* from P1's ST x.

Causality

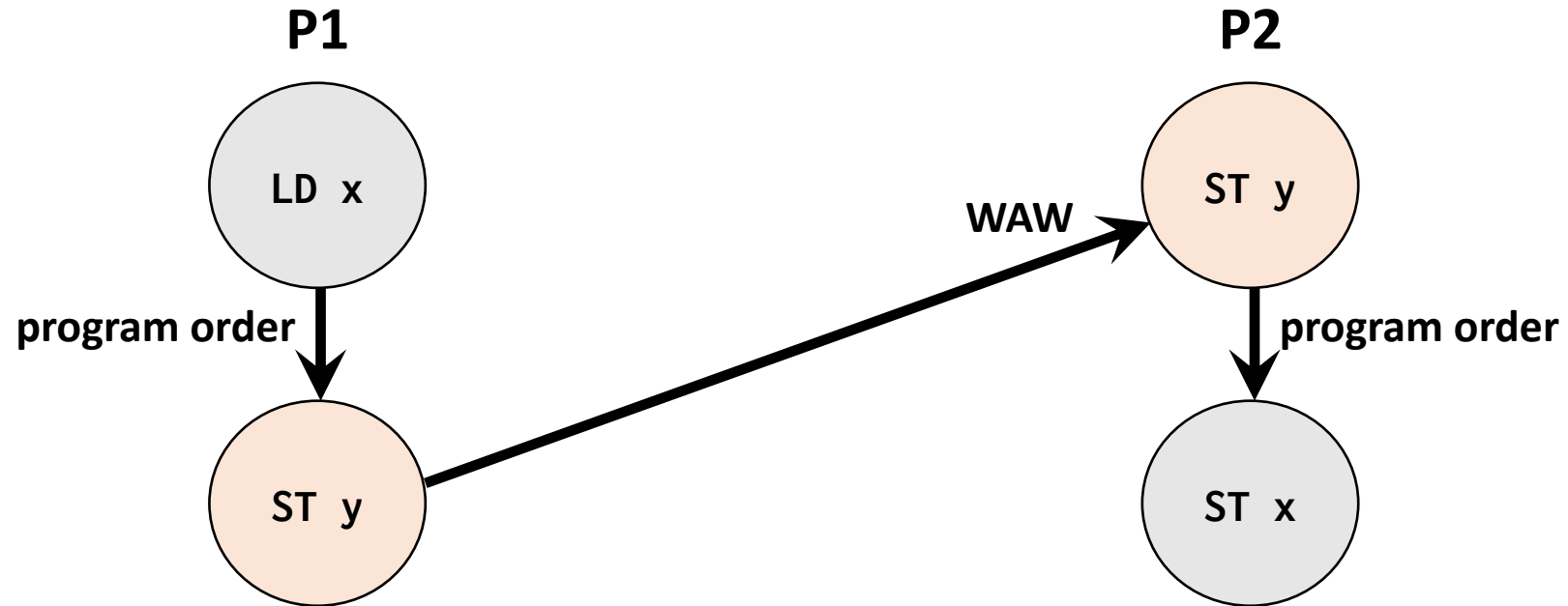


Causality

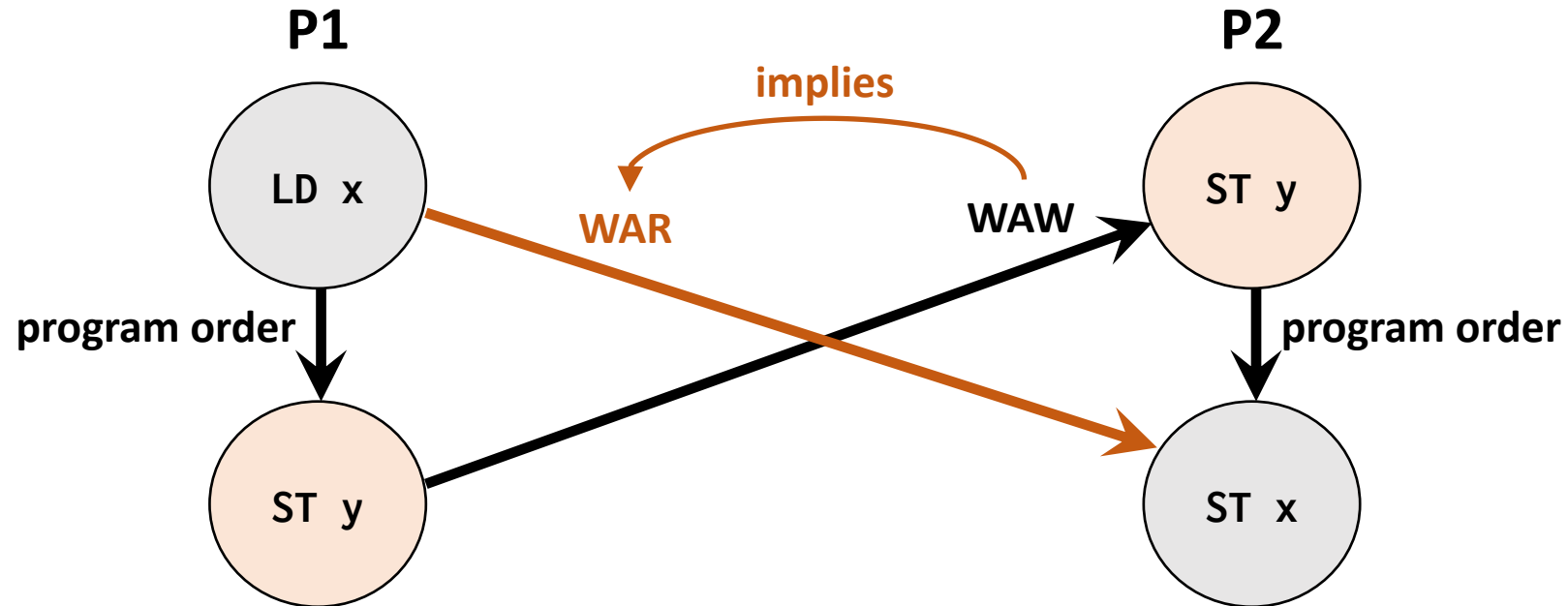


Causality implies that P2's LD x must get its value *at least* from P1's ST x.

Causality



Causality



Causality implies that P1's LD x must *not* get its value from P2's ST x.

Causality

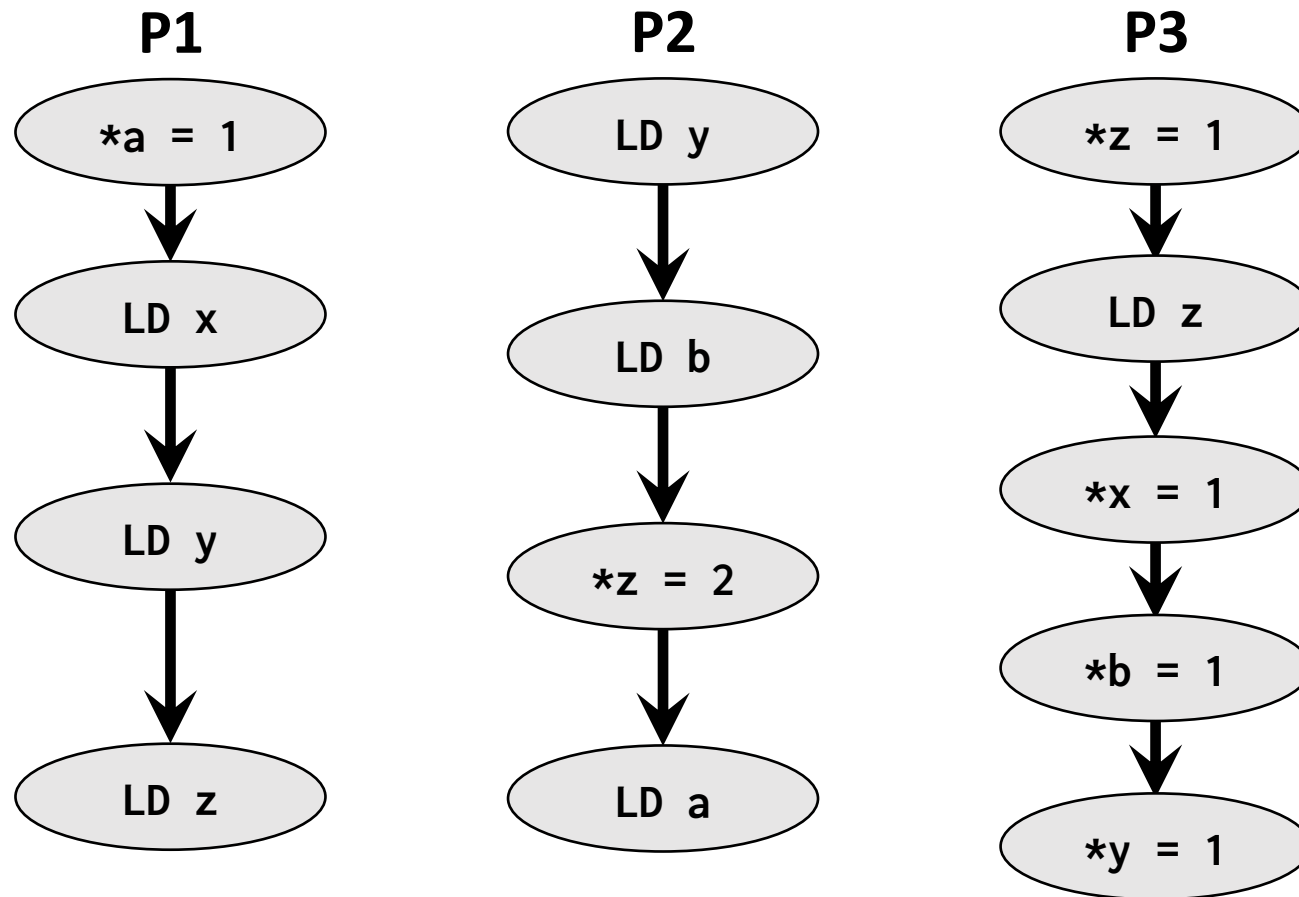
Causality extends transitively across all memory locations and all processors in system

➤ e.g., A_1 and A_2 are different values at location A

P0	P1	P2	Notes (assumes SC)
st A_1			
st B_1			
	ld B_1		Causal RAW
st A_2	st C_1		
		ld C_1	Causal RAW
	st B_2	ld $A_?$	Implies A_1 (A_2 OK also)
		ld $B_?$	Implies B_1 (B_2 OK also)

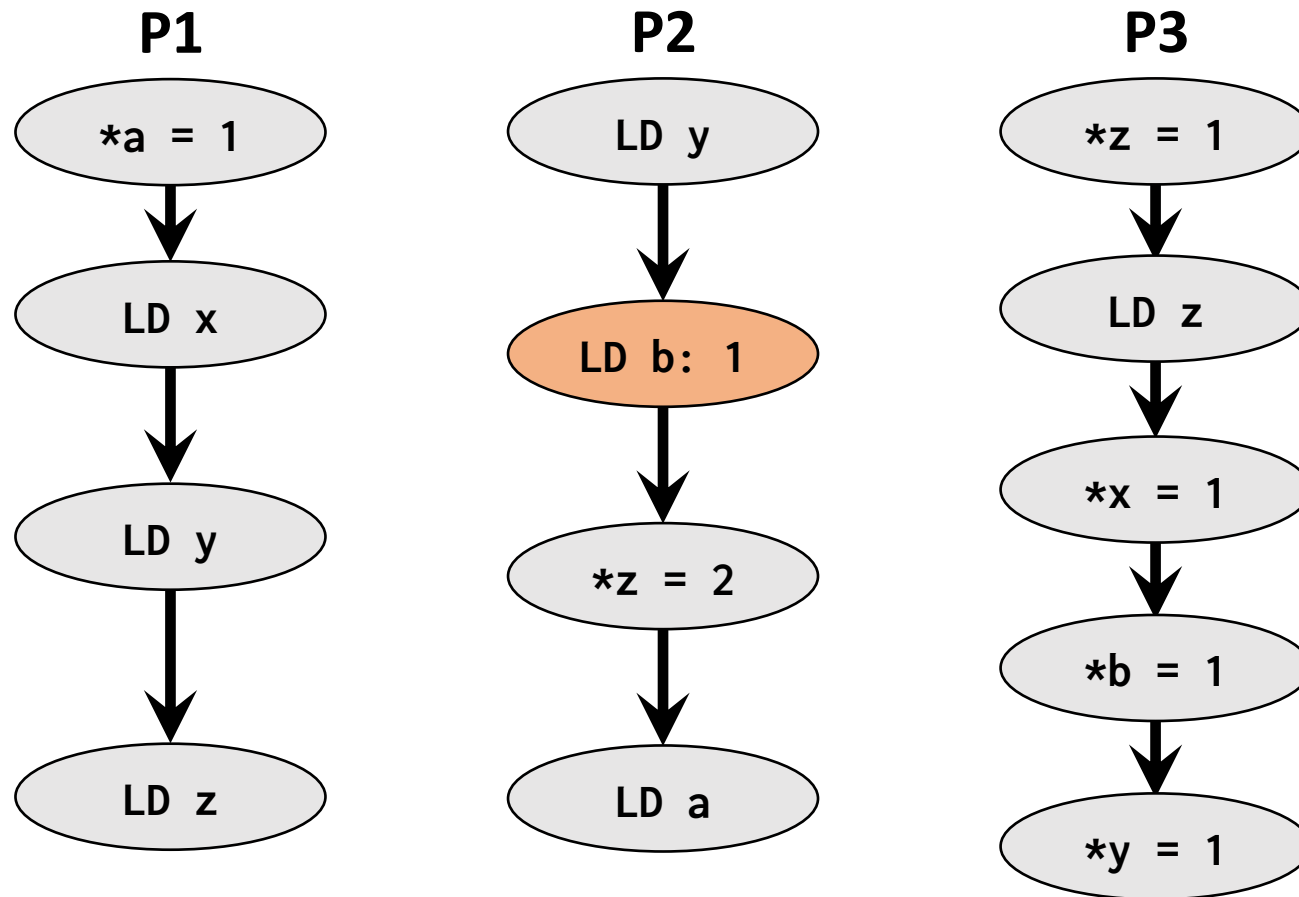
Causality

```
*x = 0; *y = 0; *z = 0; *a = 0; *b = 0; // init
```



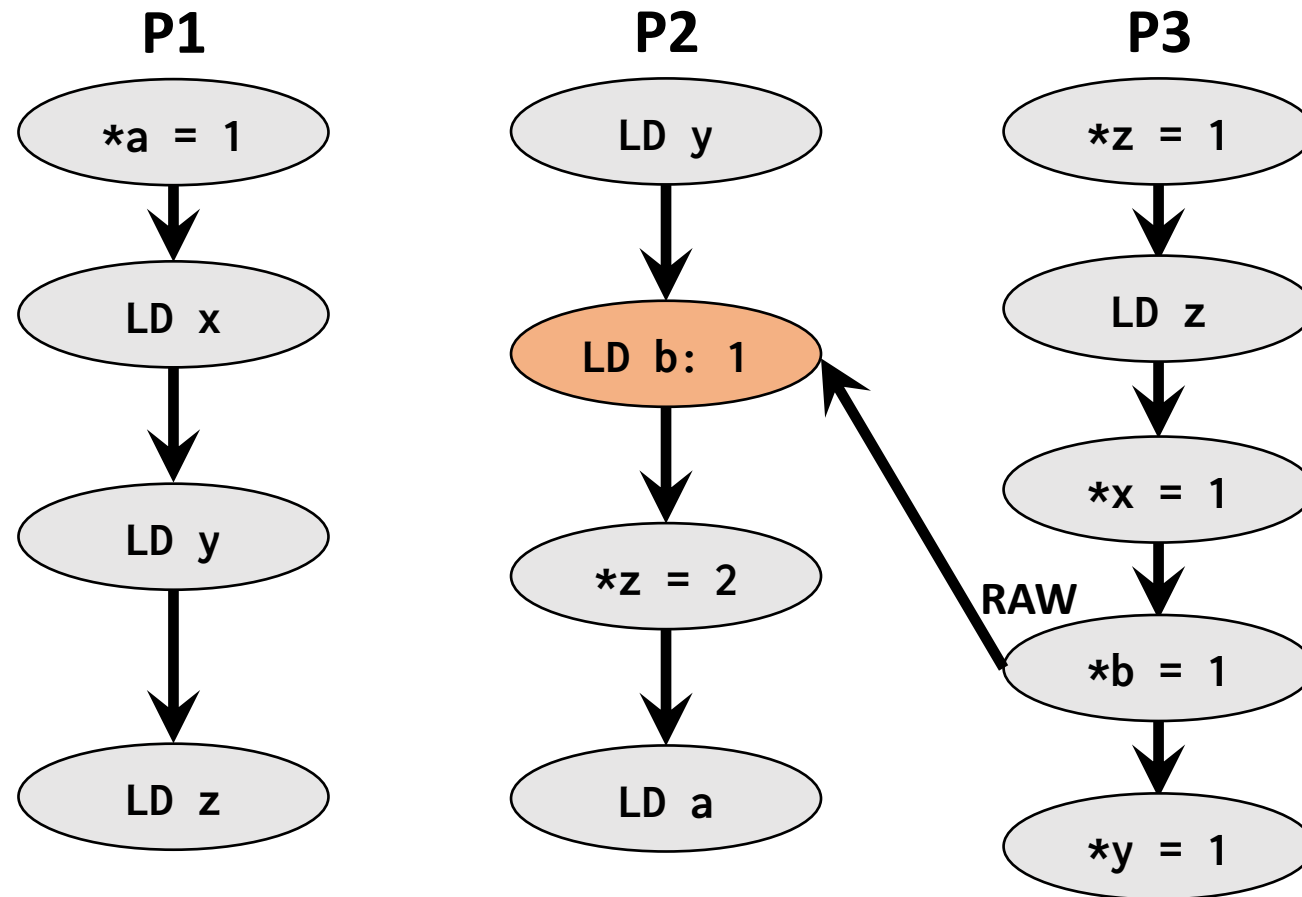
Causality

```
*x = 0; *y = 0; *z = 0; *a = 0; *b = 0; // init
```



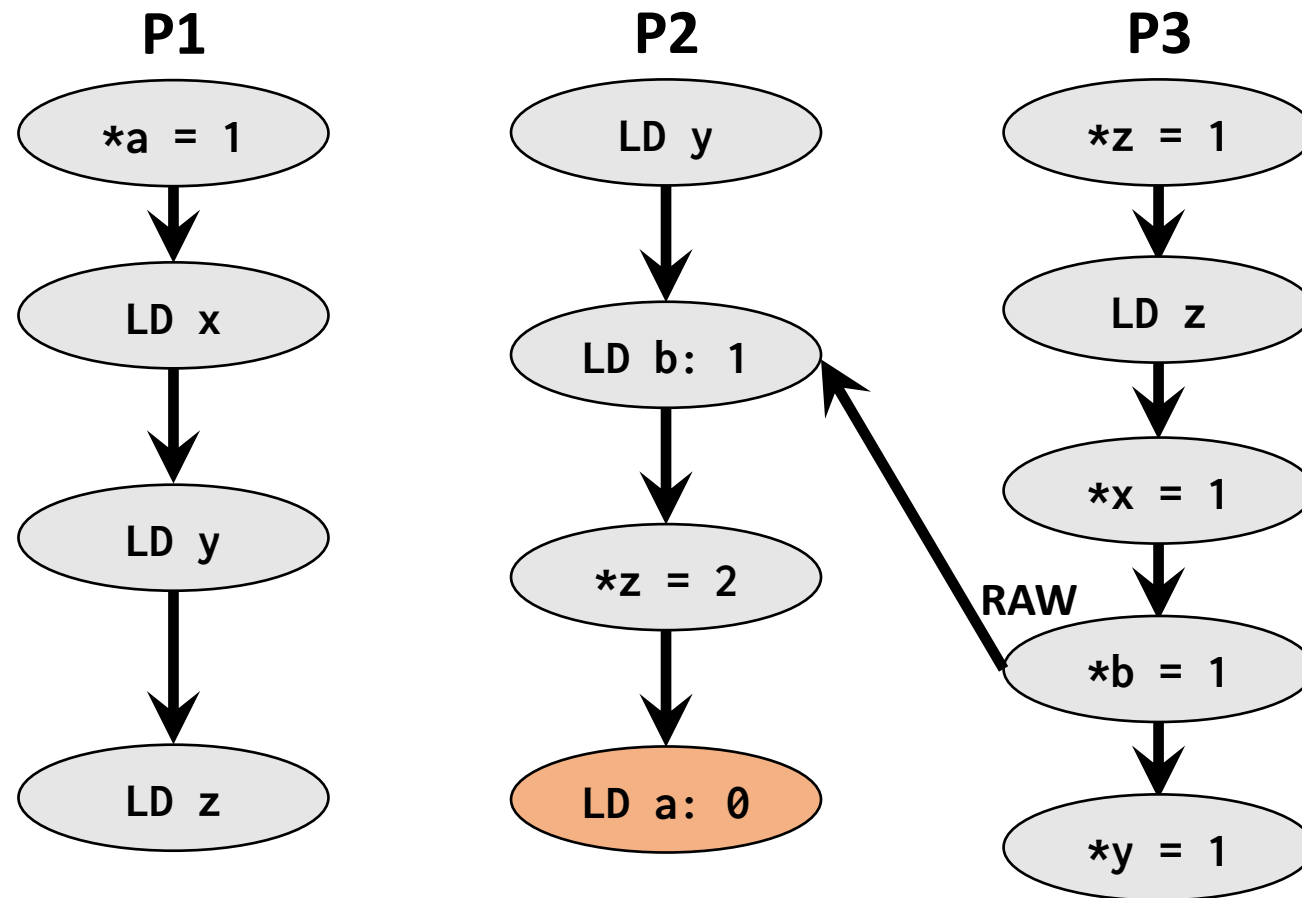
Causality

```
*x = 0; *y = 0; *z = 0; *a = 0; *b = 0; // init
```



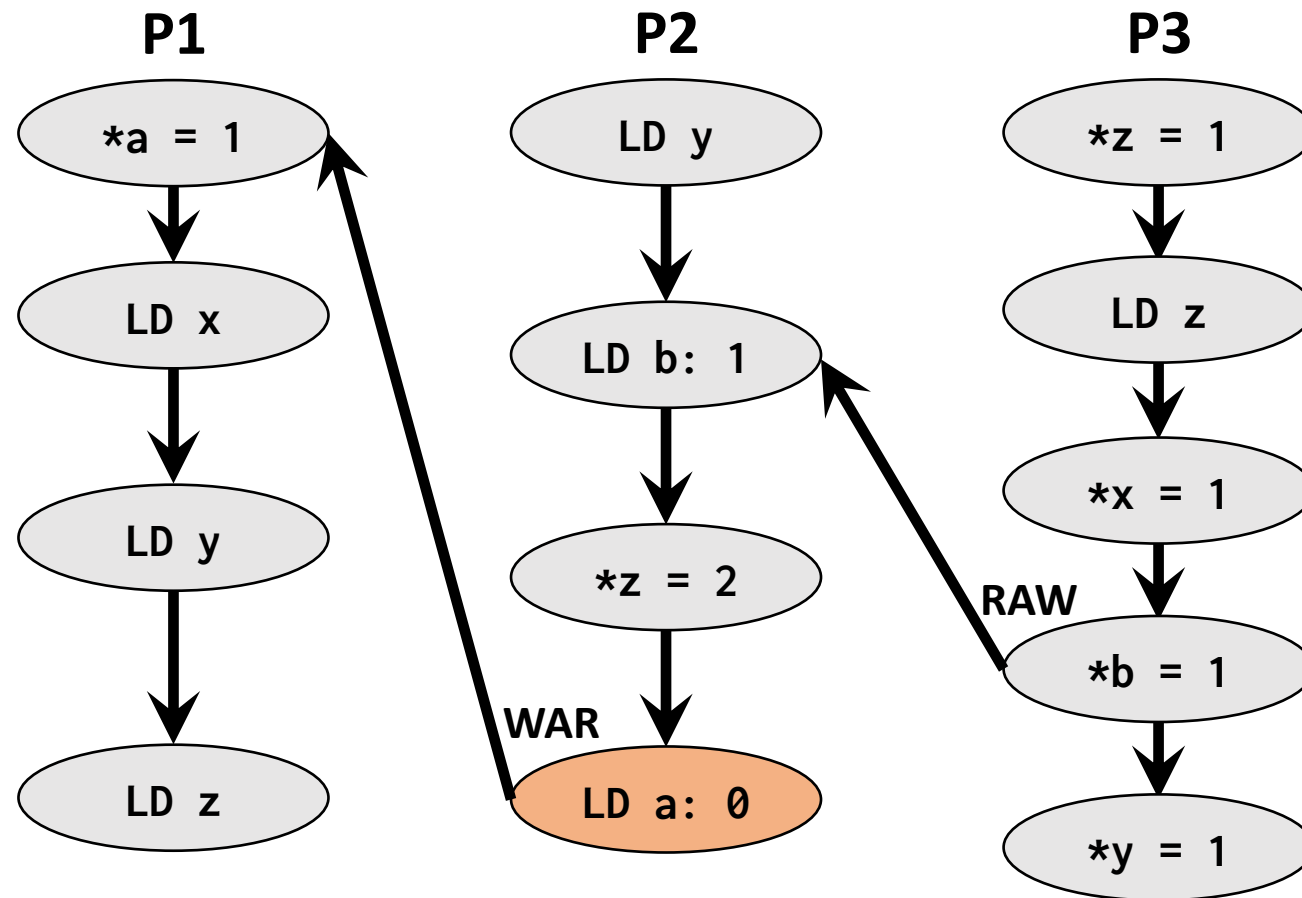
Causality

```
*x = 0; *y = 0; *z = 0; *a = 0; *b = 0; // init
```



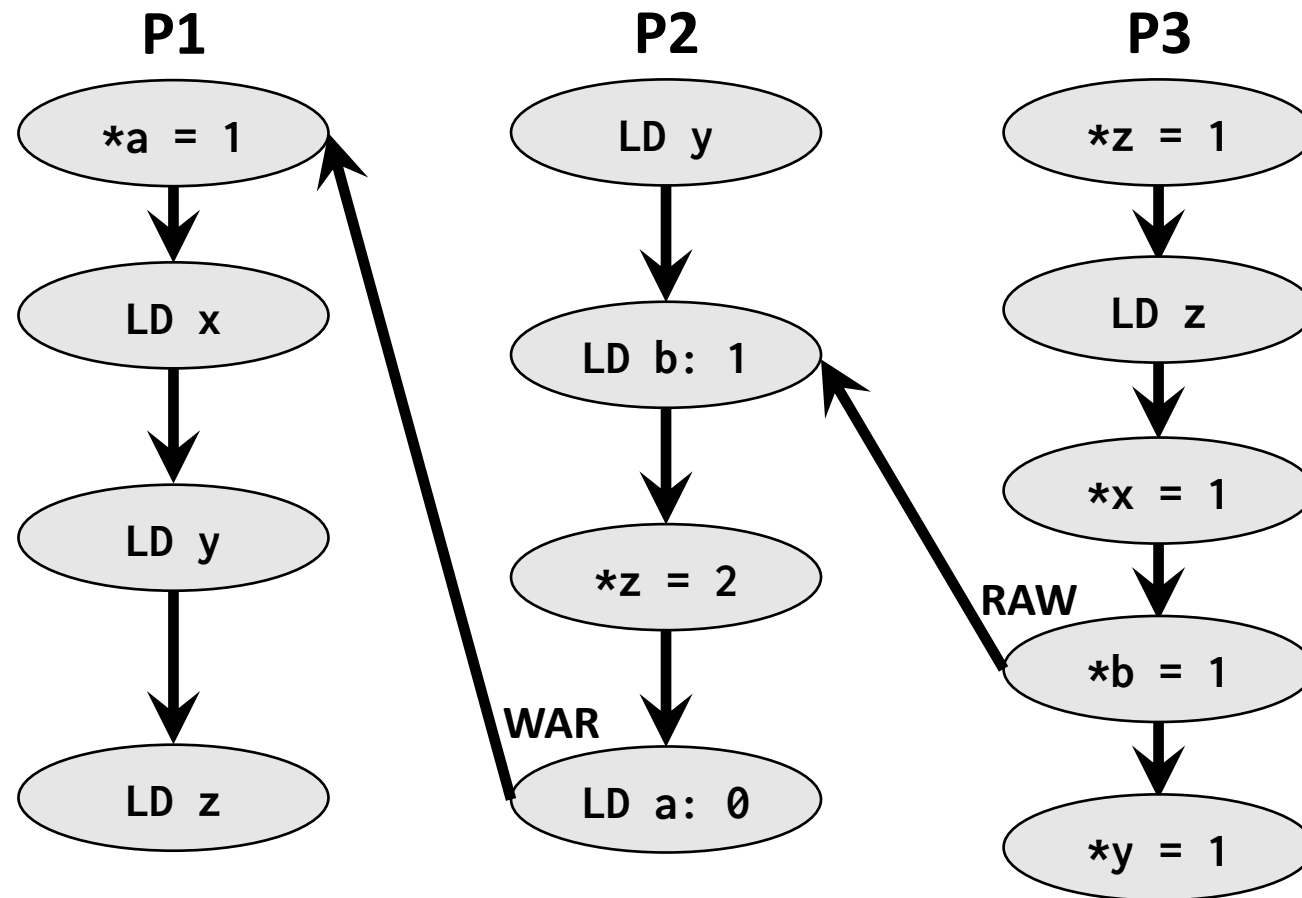
Causality

```
*x = 0; *y = 0; *z = 0; *a = 0; *b = 0; // init
```



Causality

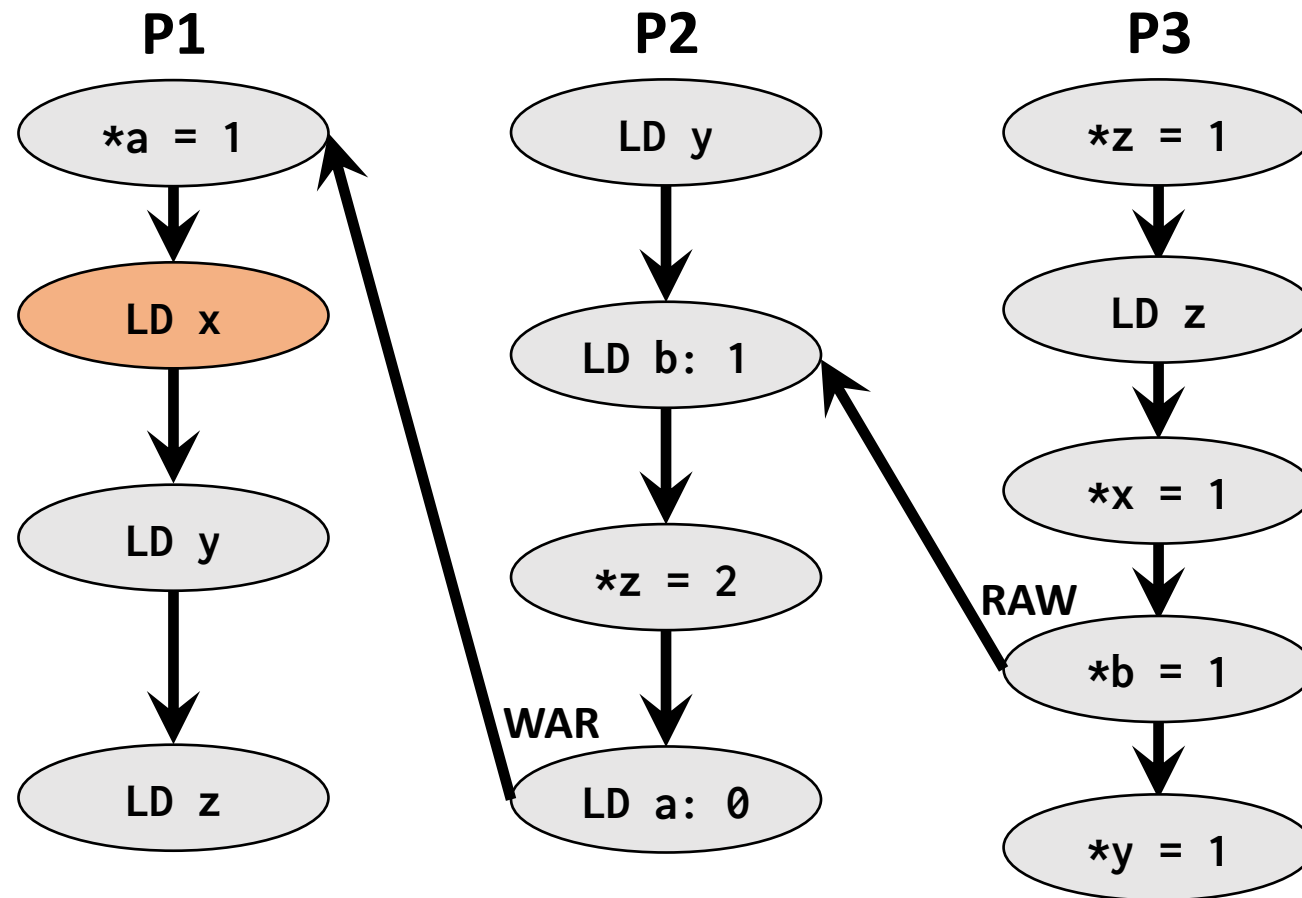
```
*x = 0; *y = 0; *z = 0; *a = 0; *b = 0; // init
```



Causality

What values must this load of x return to enforce SC?

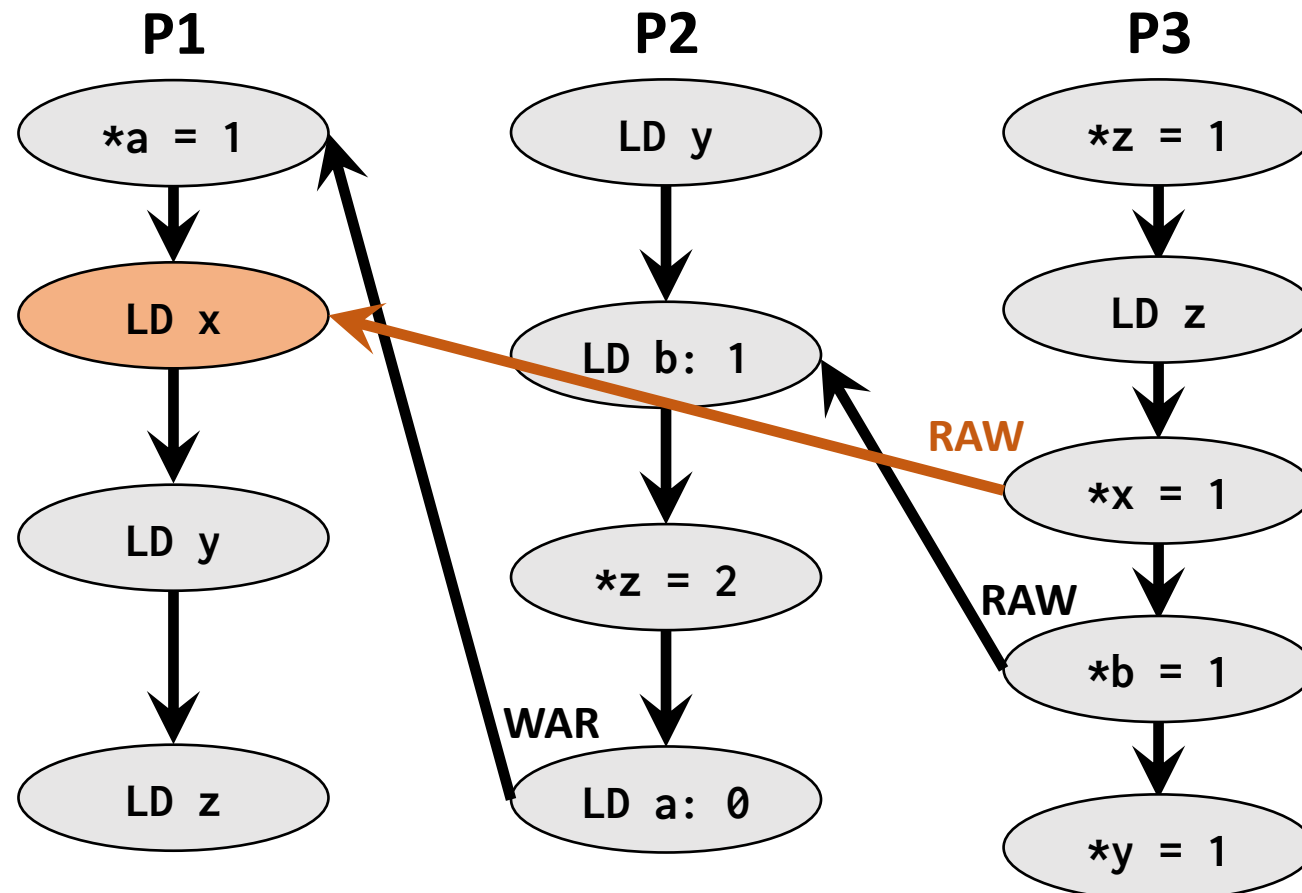
```
*x = 0; *y = 0; *z = 0; *a = 0; *b = 0; // init
```



Causality

What values must this load of x return to enforce SC? **x must be 1.**

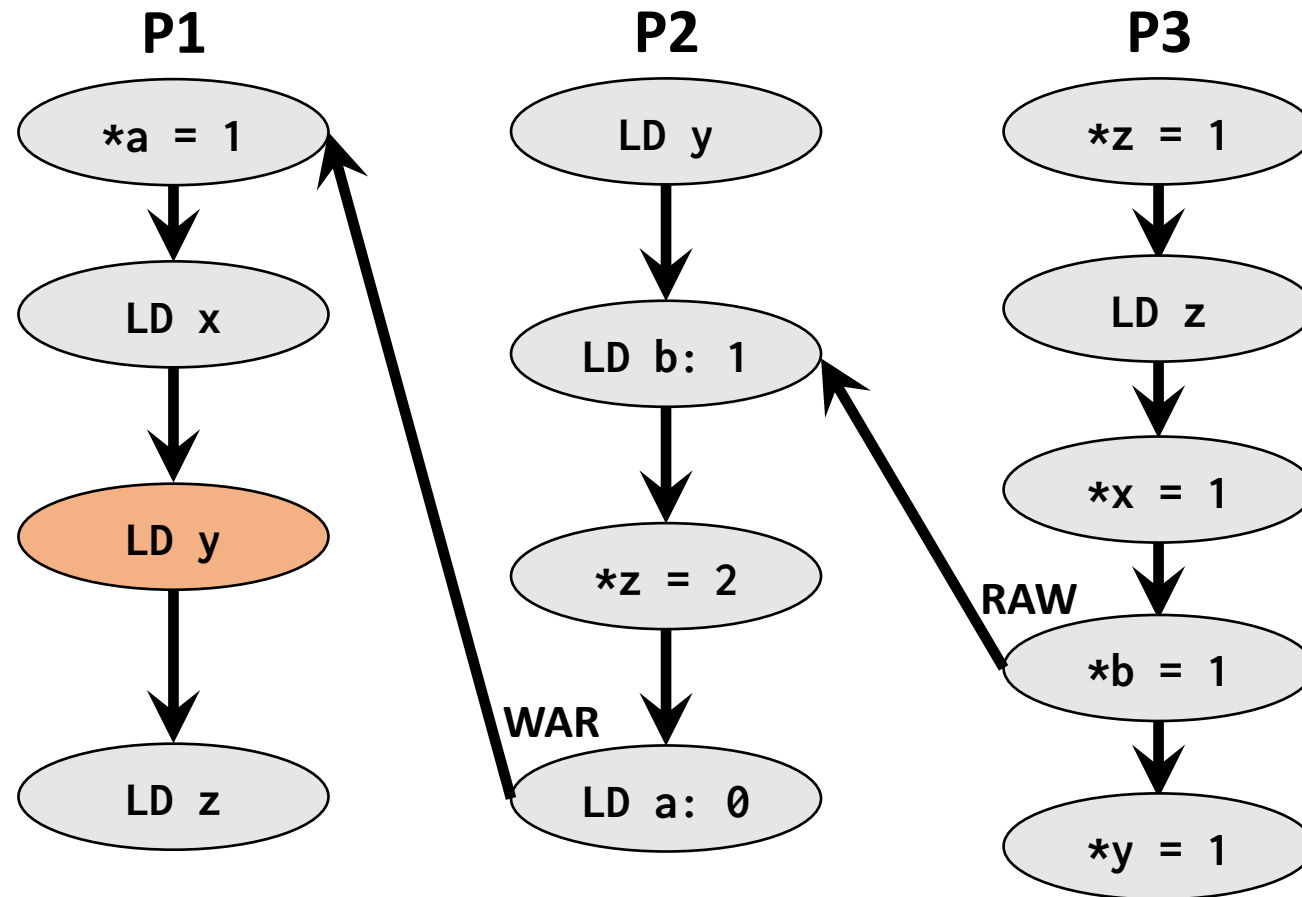
```
*x = 0; *y = 0; *z = 0; *a = 0; *b = 0; // init
```



Causality

What values must this load of *y* return to enforce SC?

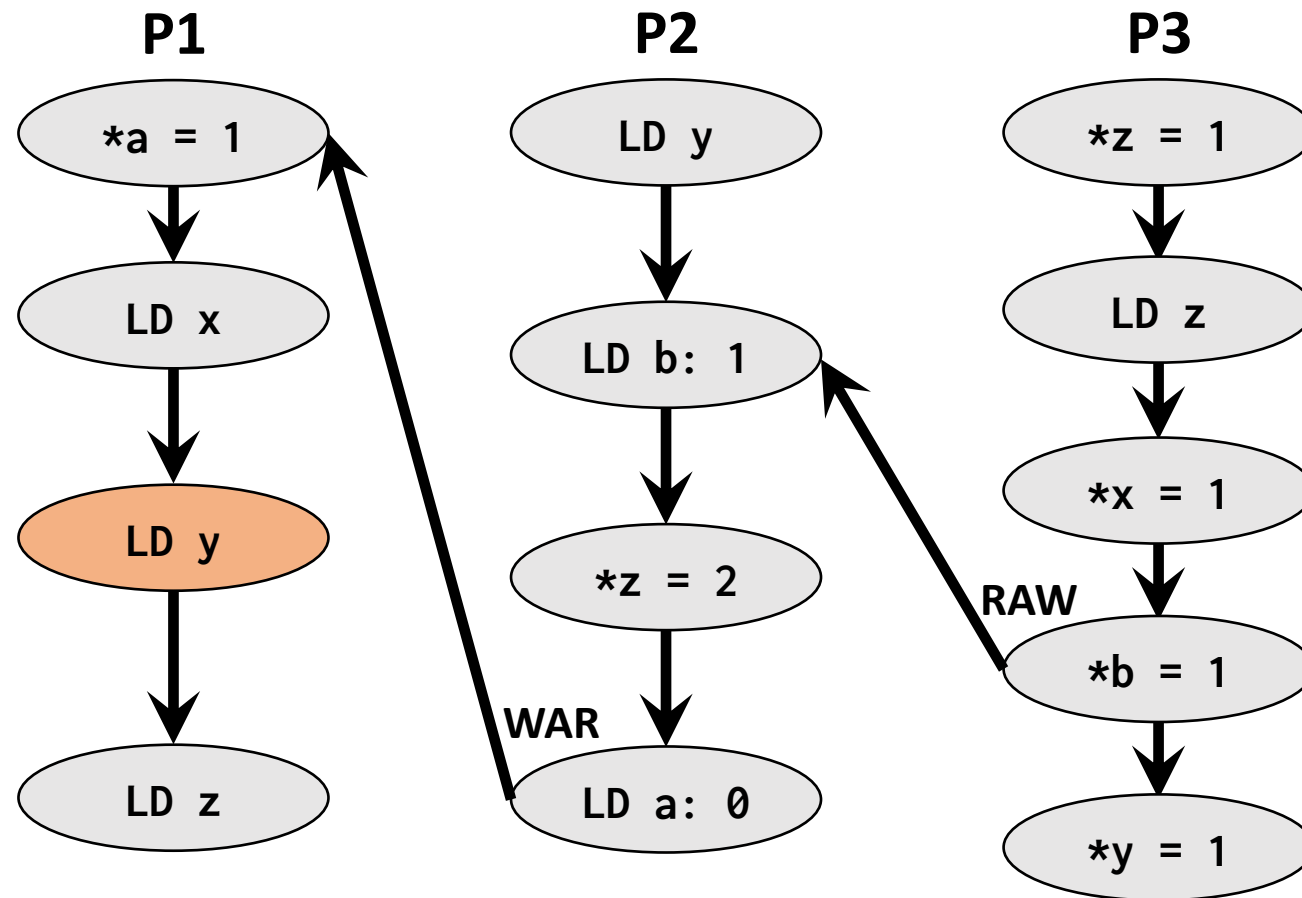
```
*x = 0; *y = 0; *z = 0; *a = 0; *b = 0; // init
```



Causality

What values must this load of *y* return to enforce SC? ***y* must be 0 or 1.**

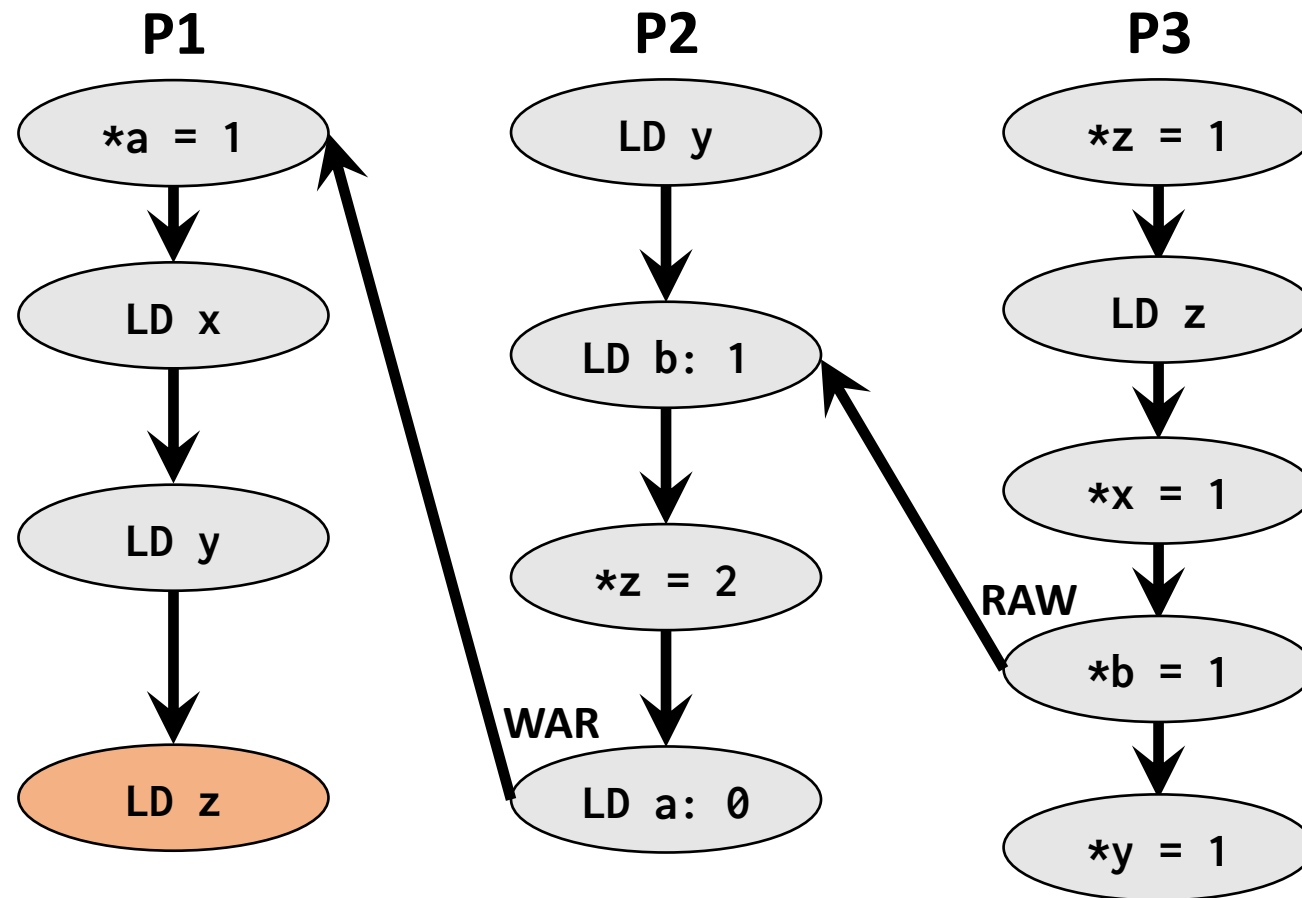
```
*x = 0; *y = 0; *z = 0; *a = 0; *b = 0; // init
```



Causality

What values must this load of z return to enforce SC?

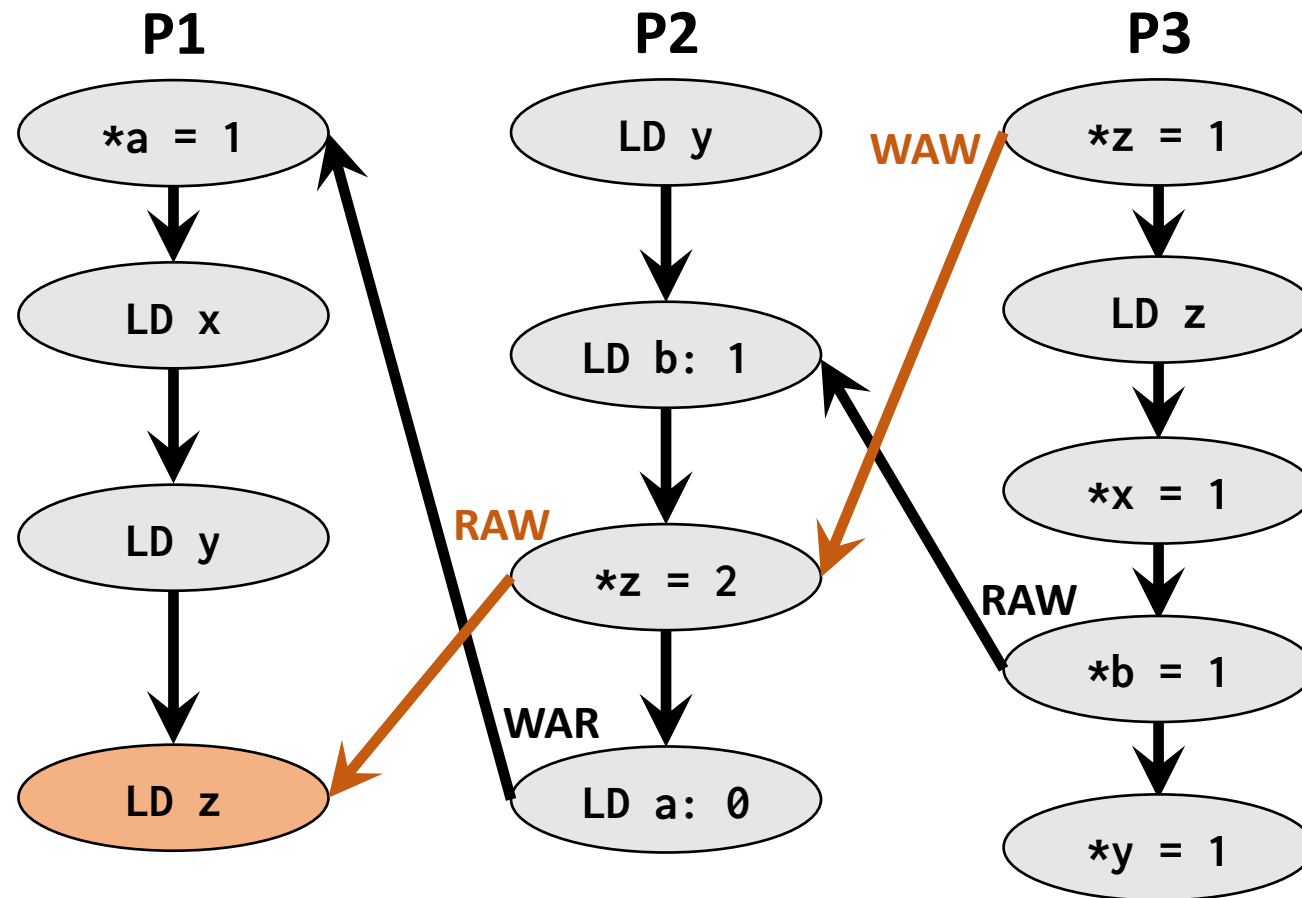
```
*x = 0; *y = 0; *z = 0; *a = 0; *b = 0; // init
```



Causality

What values must this load of z return to enforce SC? **z must be 2.**

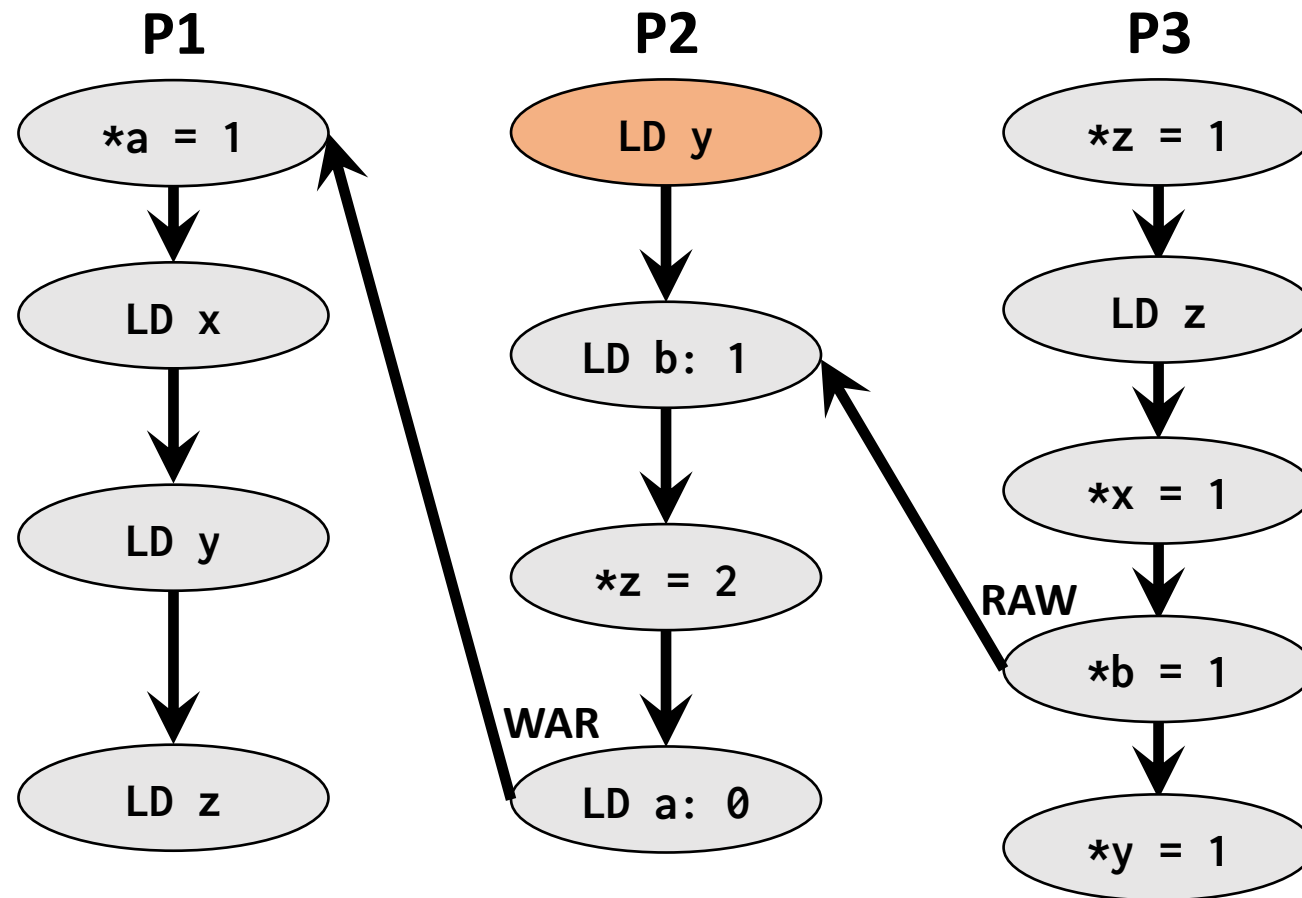
```
*x = 0; *y = 0; *z = 0; *a = 0; *b = 0; // init
```



Causality

What values must this load of *y* return to enforce SC?

```
*x = 0; *y = 0; *z = 0; *a = 0; *b = 0; // init
```



Causality

What values must this load of *y* return to enforce SC? ***y* must be 0 or 1.**

```
*x = 0; *y = 0; *z = 0; *a = 0; *b = 0; // init
```

